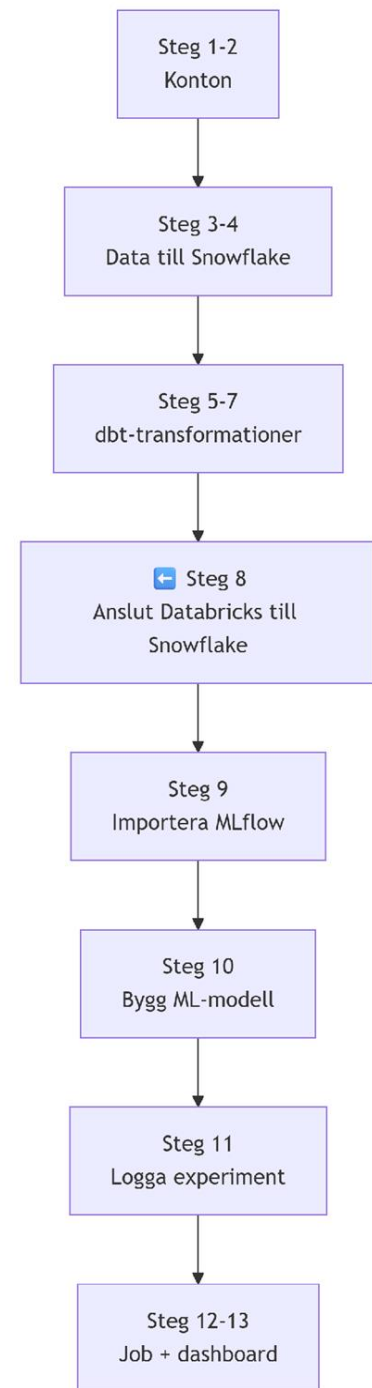


Insider Threat Detection

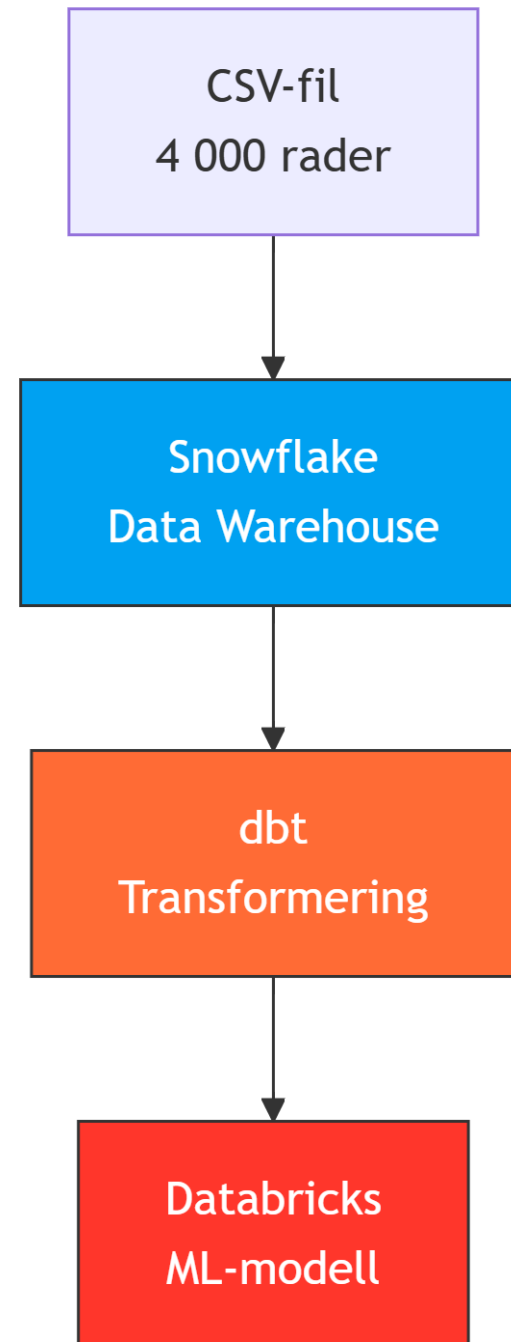
Arseniy Buzyan

Steg-för-steg-plan: Insider Threat Detection



N	Vad du gör	Var	Kommandon / Instruktioner	Kurskoppling
1	Skapa Snowflake trial-konto	Snowflake webbplats	Välj Enterprise Edition (gratis 30 dagar). Välj moln-region nära dig. Verifiera e-post.	Datainfrastruktur
2	Skapa Databricks Community Edition-konto	Databricks webbplats	Registrera dig gratis. Ingen betalkort krävs. Färdigt på 3 minuter.	Plattform för ML
3	Ladda ned Medical Internal Attack Detection Dataset	Din dator (eller Databricks notebook)	Använd Medical Internal Attack Detection Dataset från Kaggle.	Förbereda data
4	Ladda upp data till Snowflake	Databricks notebook	I Databricks: pip install snowflake-connector-python. Skriv Python-kod för att läsa CSV och ladda till Snowflake-tabell.	Data warehouse
5	Skapa dbt-projekt	Databricks Job (dbt task)	Gå till Jobs → Add task → Type: dbt. Välj Git-repo eller Workspace.	Transformationsverktyg
6	Skapa dbt-modeller för feature engineering	dbt-projektet (i Git eller Workspace)	<code>`models/sources.yml`</code> Definierar var rådatan finns i Snowflake <code>`models/staging/stg_raw_logs.sql`</code> Rensar och standardiserar rådatan (EN staging-modell) <code>`models/fct_features.sql`</code> Skapar färdiga features för ML (ingen JOIN – säker)	Förbereda för ML
7	Kör dbt-transformationer	dbt Cloud IDE (Commands)	Skriv: dbt run – skapar tabeller/views i Snowflake.	Automatisering
8	Anslut Databricks till Snowflake	Databricks notebook	Installera snowflake-connector-python. Skapa anslutning med account, user, password, warehouse, database, schema. Läs in FCT_FEATURES.	Dataförberedelse för ML
9	Importera MLflow i Databricks	Databricks notebook	import mlflow och from mlflow.tracking import MlflowClient	MLOps - experiment
10	Bygg Isolation eller Random Forest-modell	Databricks notebook	from sklearn.ensemble import IsolationForest model = IsolationForest(contamination=0.1) model.fit(df_features) eller Random Forest	ML för mönsterdetektering
11	Logga experiment med MLflow	Databricks notebook	mlflow.set_experiment("/Users/din_email/insider_threat") with mlflow.start_run(): mlflow.log_param("contamination", 0.1) mlflow.log_metric("anomalies", n_anomalies) mlflow.sklearn.log_model(model, "model")	MLOps - spårning
12	Skapa Databricks Job för automatisering	Databricks Jobs	Skapa nytt Job. Lägg till notebook med ML-modellen. Sätt schema (t.ex. dagligen).	AI för detektering
13	Bygg enkel dashboard i	Databricks SQL	Skapa SQL-fråga: SELECT user_id, COUNT(*) as anomalies FROM	Övervaka system

Bakgrund:
Varför denna
arkitektur
(dbt →
Snowflake →
Databricks)?



Varför inte
bara ladda
CSV direkt i
Databricks
?

Fråga	Svar
Kan man ladda CSV direkt i Databricks?	 Ja, det går!
Varför gör man inte det i produktion?	 För att CSV är opålitligt för stora mängder data

Enkelt exempel från verkligheten:

Scenario	CSV direkt	Snowflake + dbt
Du får ny data varje dag	Du måste ladda upp CSV manuellt varje dag	Snowflake uppdateras automatiskt
Flera personer jobbar med samma data	Alla har sin egen kopia (olika versioner)	Alla ser SAMMA data
Data är 100 GB (inte 308 KB)	Databricks kraschar eller blir extremt långsam	Snowflake hanterar TB utan problem
Du behöver rensa datan	Du skriver ny kod varje gång	dbt återanvänder samma SQL-modeller

Databricks vs Jupyter vs Google Colab

Funktion	Databricks (produktion)	Jupyter (lokal)	Google Colab (gratis moln)
Skalbarhet	✓ Hanterar TB data	✗ Bara din dators RAM	⚠ Begränsad RAM/CPU
Samarbete	✓ Flera personer samtidigt	✗ Dela filer manuellt	⚠ Dela notebooks, men inte data
Schemaläggning	✓ Automatiska jobb (cron)	✗ Nej	✗ Nej
Inbyggd MLflow	✓ Ja (gratis)	✗ Kräver installation	✗ Kräver installation
Spark (stordata)	✓ Inbyggt	✗ Kräver installation	✗ Kräver installation
Kostnad	Community Edition gratis	Gratis	Gratis
Används i företag	✓ Airbnb, Spotify, etc.	⚠ För prototyping	✗ Sällan

Verkligt exempel

▪
▪



Spotify använder
Databricks för att analysera
miljarder musikstreams.

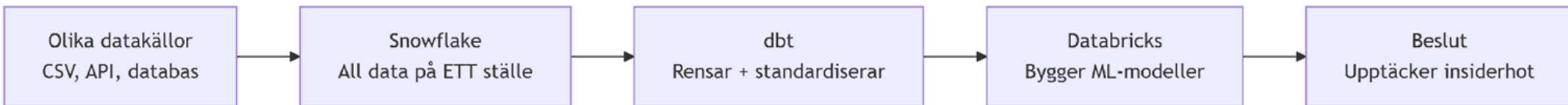


Jupyter används av en
dataforskare hemma på sin
laptop.



Google Colab används för
utbildning och snabba
experiment.

Varför produktion använder dbt + Snowflake + Databricks



Lager	Verktyg	Varför?	Verkligt exempel
Data Warehouse	Snowflake	All företagsdata på en plats, säker, skalbar	Alla kunders köp från alla butiker
Transformering	dbt	Samma rensningslogik återanvänds av alla team	"misslyckade inloggningar" räknas likadant överallt
ML-plattform	Databricks	Hanterar stora datamängder, automatiserar jobb	Tränar en ny modell varje natt med nya data

Sammanfattning med en pizzabakelse:

•**Varför inte bara lägga allt i ugnen direkt?** För att du vill ha receptet (dbt) så att du kan baka samma pizza varje dag, och kylskåpet (Snowflake) så att ingredienserna inte blir förstörda.

•**Varför Databricks istället för Jupyter?** För att i produktion bakar du pizzor till 10 000 kunder samtidigt – din laptop orkar inte. Databricks är som en fabriksugn som gör allt automatiskt varje natt.

N	Verktyg	Liknelse
1	CSV (rådata)	Råa ingredienser (mjöl, tomater, ost)
2	Snowflake	Kylskåpet – förvarar allt säkert
3	dbt	Receptet – säger exakt hur man gör
4	Databricks	Ugnen – bakar pizzan (bygger ML-modellen)

Verktyg du behöver (alla gratis)

Verktyg	Syfte	Kostnad
Snowflake	Lagring av rådata och features	30-dagars trial (Enterprise Edition)
Databricks Community Edition	Notebooks, ML, Jobs, MLflow	Permanent gratis (begränsad men räcker)
dbt Core (via Databricks)	Transformationer	Gratis, körs i Databricks
MLflow (inbyggt)	Experiment tracking	Gratis i Databricks CE
GitHub	Versionshantering, portfolio	Gratis

Steg 1:

Skapa Snowflake Trial-konto (Steg-för-steg)

N	Åtgärd	Detaljerade Instruktioner	Skärmdump / Notering
1.1	Gå till rätt webbplats	Öppna din webbläsare och gå till: https://trial.snowflake.com	Viktigt: Använd denna länk för att komma direkt till rätt registreringsformulär.
1.2	Fyll i dina uppgifter	Ange följande information: • Full name: Ditt för- och efternamn. • Email: Din e-postadress (Gmail/Outlook fungerar utmärkt) . • Role: Välj Data Engineer eller Student. • Company: Skriv Self Employed eller Student .	Snowflake kräver inga betalkortsuppgifter för att starta en trial .
1.3	Väl rätt "Edition"	Välj Enterprise Edition .	Varför? Denna version ger dig tillgång till alla funktioner du behöver för projektet, till exempel Materialized Views, som inte finns i Standard Edition .
1.4	Välj molnleverantör & region	• Cloud Provider: Välj Microsoft Azure (fungerar lika bra som AWS/GCP). • Region: Välj den region som är geografiskt närmast dig (t.ex. Sweden Central eller North Europe).	Detta minskar eventuell fördröjning. Du ärver ingen kostnad för ditt val, det är bara för att optimera hastigheten.
1.5	Bekräfta att du är människa	Klicka i alla rutor som visar cyklar (eller det föremål som visas på skärmen) för att verifiera att du inte är en robot .	Läs instruktionen noga. Ibland ber den dig klicka på bilar eller skyltar.
1.6	Svara på frågor (valfritt)	Du kommer att få några frågor om din erfarenhet. Svara ärligt eller hoppa över. Detta påverkar inte ditt konto eller dess krediter.	Du kan till exempel välja "Cloud Data Warehousing" som intresseområde.
1.7	Aktivera ditt konto via e-post	Gå till din e-postinkorg. Du kommer att få ett mejl från Snowflake med en aktiveringslänk ("CLICK TO ACTIVATE") .	Spara länken! Detta är din unika URL till ditt Snowflake-konto. Den kommer att se ut ungefär så här: https://xxxxx.ap-southeast-2.azure.snowflakecomputing.com
1.8	Skapa dina inloggningsuppgifter	Klicka på aktiveringslänken. Du kommer då till en sida där du skapar ditt användarnamn och lösenord .	Kom ihåg dessa! Du kommer att behöva dem varje gång du loggar in.
1.9	Logga in för första gången	Efter att du skapat ditt lösenord klickar du på "Get Started" eller "Sign In". Du är nu inne i Snowflake-gränssnittet!	Välj "Skip" om den frågar om du vill sätta upp exempeldata. Vi kommer att ladda upp vår egen data senare.

De viktigaste delarna för DIG (resten kan du ignorera för nu)

Vad du ser	Vad det betyder	Ska du använda det?
\$400 credits left	Din gratis budget – räcker till massor av analyser	✓ Bra – pengar kvar!
Trial ends in 30 days	Din period räknar ner från idag	✓ Normalt – håll koll
Upload local files	Ladda upp CSV/Excel-filer från din dator	✓ JA – här börjar du!
Worksheets	Skriva SQL-kod (som ett anteckningsblock)	✓ JA – här gör du arbetet
Databases (i vänstermenyn under "Catalog")	Var din data lagras	✓ JA – dina tabeller syns här
Compute (vänstermenyn)	Din "motor" som kör beräkningar	✓ JA – starta/pausa här
Postgres	En databastyp (inte relevant för dig just nu)	✗ Ignorera
Horizon Catalog	Ett nytt sätt att organisera data	✗ Ignorera för nu
Marketplace	Köpa färdiga dataset (du behöver inte)	✗ Ignorera
All projects / Files / Notebooks /	Avancerade funktioner	✗ Ignorera – använd

Checklista innan Steg 2.

Mål: Verifiera att Snowflake-kontot är korrekt inställt och redo för nästa steg (Databricks + dbt + ML)

N	Kategori	Vad ska du göra	Exakt handling	Varför (kort)
1	Roll-kontroll	Byt till ACCOUNTADMIN	Högst ner till vänster → klicka på din nuvarande roll → välj ACCOUNTADMIN från listan	Ger dig alla rättigheter – annars kan du inte skapa tabeller eller ladda data
2	Compute-motor	Starta ditt virtual warehouse	Gå till Compute i vänstermenyn → hitta COMPUTE_WH → klicka "Start" / "Resume"	Du behöver en motor för att köra SQL och ladda data
3	Verifiera anslutning	Kör ett test-SQL-kommando	Gå till Worksheets → skapa ny worksheet → skriv SELECT CURRENT_VERSION(); → klicka "Run"	Bekräftar att allt fungerar och att du är uppkopplad
4	Kontrollera krediter	Notera din återstående budget	Titta högst upp: \$400 credits left	Håll koll så att du inte överskrider (kom ihåg att pausa din warehouse när du är klar)
5	Kontrollera trial-period	Notera slutdatum	Titta högst upp: Trial ends in X days	Planera ditt projekt så att du hinner klart inom 30 dagar

Quick actions – Vad du ska göra VARJE GÅNG du arbetar

När	Handling	Var	Kort kommando / knapp
Innan du börjar	Starta din warehouse	Compute → COMPUTE_WH	Klicka " Start " / " Resume "
Under arbetet	Kör SQL-kod	Worksheets	Skriv SQL → klicka " Run "
Innan du slutar	Pausa din warehouse	Compute → COMPUTE_WH	Klicka " Suspend "
Vid problem	Kontrollera din roll	Högst upp till höger	Ska stå ACCOUNTADMIN
Vid felmeddelande	Starta om worksheet	Worksheets	Ladda om sidan (F5) eller skapa ny worksheet

Pro tips

#	Pro tip	Förklaring
1	Pausa alltid din warehouse när du är klar	Du betalar bara när den är igång. Att vara inloggad kostar inget.
2	Använd ACCOUNTADMIN-rollen hela tiden	Det är enklast för nybörjare. Andra roller har begränsningar.
3	Spara din Snowflake-URL	Den ser ut som https://xxxxx.azure.snowflakecomputing.com – spara den i dina bokmärken.
4	Du kan skapa nytt trial-konto med samma email	När 30 dagar är slut – bara registrera om. Dina krediter återställs.
5	Undvik att ladda upp stora filer	Håll dig under 100 MB per fil – annars går det långsamt i trial-versionen.
6	Skriv kommentarer i din SQL-kod	Använd <code>--</code> för att skriva kommentarer. Ex: <code>-- Detta skapar en tabell för loggar</code>

Steg 2: Skapa Databricks Community Edition-konto. Databricks Community Edition är helt gratis, kräver inget betalkort, och är permanent (ingen 30-dagarsgräns som Snowflake)

N	Åtgärd	Exakt länk / Var	Detaljerade instruktioner
2.1	Gå till rätt sida	https://community.cloud.databricks.com	Öppna länken i din webbläsare. Detta är den enda vägen till Community Edition.
2.2	Klicka på "Sign Up"	Första sidan	Leta efter knappen "Sign up" eller "Try Databricks"
2.3	Fyll i dina uppgifter	Registreringsformulär	Ange: Namn, E-post (Gmail/Outlook fungerar bäst) , Företag (skriv "Student" eller "Self Employed")
2.4	Välj rätt alternativ – VIKTIGAST!	Välj molnleverantör-sida	Scrolla ner – klicka på den lilla textlänken "Get started with Community Edition" . Klicka INTE på de stora AWS/Azure/GCP-knapparna!
2.5	Bekräfta valet	Pop-up-fönster	Klicka "OK" eller "Confirm" på meddelandet "Community Edition has limited features"
2.6	Kontrollera din e-post	Din e-postinkorg (Gmail/Outlook)	Du får ett verifieringsmejl från Databricks inom 2 minuter . Kolla skräpposten om det inte syns!
2.7	Verifiera din e-post	Klicka på länken i mejlet	Länken heter "Verify email address" eller liknande . Du skickas till en sida där du skapar ditt lösenord.
2.8	Skapa lösenord	Lösenordssidan	Välj ett starkt lösenord. Klicka "Submit"
2.9	Logga in	https://community.cloud.databricks.com	Använd din e-post och ditt nya lösenord
2.10	Kolla att du är på rätt ställe	Databricks startsida	Du ser ett gränssnitt med vänstermeny: Workspace, Compute, Data, Workflows

Vanliga fallgropar – och hur du undviker dem

Problem	Varför händer det?	Lösning
Jag ser bara AWS/Azure/GCP-alternativ	Du klickade inte på den lilla textlänken	Gå tillbaka, scrolla ner på sidan, klicka på "Get started with Community Edition"
Jag får ingen verifierings-e-post	Skräppostfilter eller fel e-post	Kolla skräpposten. Använd Gmail/Outlook istället för jobbmejl
Jag hamnade på 14-dagars trial	Du klickade på fel knapp	Börja om från steg 2.1. Community Edition är permanent gratis
Sidan ber om betalkort	Du är på fel sida	Gå till community.cloud.databricks.com – inte vanliga Databricks-sidan

Frågor från registrering

What do you want to do first?

 **"See what Databricks can do"**

Tell us your goal

 **"Try demos and samples to understand real world workflows"**

Ditt val

Data Warehousing

ETLs and Pipelines

Lakehouse Fundamentals

Koppling till ditt projekt

Snowflake + Databricks jobbar med detta

dbt-transformationer = ETL

Grunden för hur Databricks fungerar

Varför Community Edition (och inte 14-dagars trial)?

Funktion	Community Edition	14-dagars Trial
Kostnad	Permanent gratis	Gratis i 14 dagar
Betalkort	✗ Behövs inte	✓ Krävs
Molnkonto (AWS/Azure)	✗ Behövs inte	✓ Behövs
Passar för	Studenter, hobbyprojekt	Företag, POC
Ditt projekt	✓ Perfekt!	✗ För kort tid

De viktigaste delarna för DIG (resten kan du ignorera för nu)

Vad du ser

"Welcome to Databricks"

"Get started"

"Data engineer" / "Data

Du ser detta

Start learning

Learn the basics

Watch videos

Explore demos

Betyder

☒ Inloggad korrekt

☒ Gratisversionen

☒ Community Edition

Ska du göra?

☐ Nej

☐ Nej

☐ Nej

☐ Nej

☒ Ja

Steg 3.

Ladda ned Medical Internal Attack

Detection Dataset:

<https://www.kaggle.com/datasets/zara2099/medical-internal-attack-detection-dataset?resource=download>

•Datasetet innehåller information om:

- num_failed_logins – misslyckade inloggningsförsök
- num_file_creations – antal skapade filer
- num_access_files – antal åtkomster till patientjournaler
- attack_type – klassificering av normal användning eller attack

•Detta är **exakt insider threat-beteende** som du ska analysera med din ML-modell

EGENSKAP VÄRDE		VARFÖR BRA
Filstorlek	308.65 kB	✓ Laddas ner på sekunder
Antal rader	4 000 rader	✓ Lagom storlek för ML
Antal kolumner	15 kolumner	✓ Enkelt att förstå
Innehåll	Loggar över användarbeteende	✓ Perfekt för insider threat-detektering
Målkolumn	attack_type	✓ Används för att träna ML-modellen

Skapa Kaggle-konto (om du inte redan har ett)

N	Åtgärd	Detaljerad instruktion	Skärmtips
3.1	Gå till Kaggle	Öppna webbläsaren och gå till www.kaggle.com	Använd Chrome, Firefox eller Edge
3.2	Klicka på "Register"	Leta efter registreringsknappen uppe till höger	Den är blå och syns tydligt
3.3	Välj registreringsmetod	Välj "Register with Email" eller använd Google-konto	Rekommenderar Google – snabbast
3.4	Fyll i dina uppgifter	Ange: Namn, E-postadress (Gmail/Outlook fungerar bäst), Lösenord	Använd en e-post du kommer åt
3.5	Verifiera e-post	Gå till din e-postinkorg och klicka på verifieringslänken från Kaggle	Kolla skräpposten om du inte ser den
3.6	Logga in	Använd din e-post och ditt lösenord för att logga in på Kaggle	Du är nu redo att ladda ner dataset

Ladda ner datasetet

Steg	Åtgärd	Detaljerad instruktion	Länk / Var
3.7	Gå till dataset-sidan	Öppna denna länk i din webbläsare:	https://www.kaggle.com/datasets/zara2099/medical-internal-attack-detection-dataset
3.8	Läs om datasetet	Scrolla ner och läs beskrivningen	Det hjälper dig förstå vad datan innehåller
3.9	Klicka på "Download"	Leta efter den blå "Download" - knappen uppe till höger på sidan	Knappen har en nedåtpil-ikon
3.10	Vänta på nedladdningen	Filen Medical_Internal_Attacks_Dataset.csv laddas ner	Tar bara några sekunder (308 KB)
3.11	Spara filen på din dator	Välj en plats du lätt hittar, t.ex. Skrivbordet eller en mapp som heter "Projektdata"	Kom ihåg var du sparar!

Vanliga frågor

Fråga

Måste jag betala för Kaggle-konto?

Svar

Nej, Kaggle är helt gratis!

Varför just detta dataset?

Det är litet (308 KB), laddas ner snabbt, och innehåller perfekt data för insider threat-detektering.

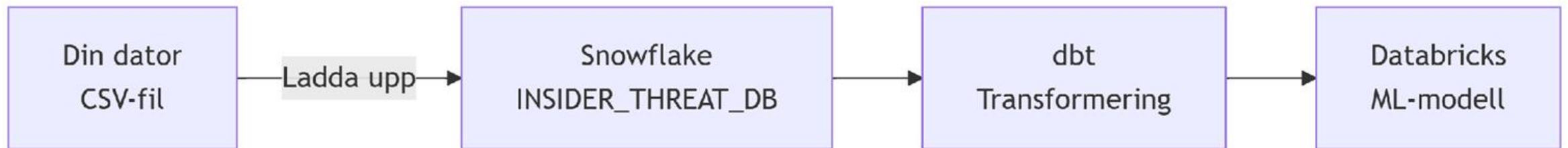
Kan jag använda ett annat dataset?

Ja, men detta är rekommenderat för nybörjare eftersom det är enkelt och litet.

Vad gör jag om nedladdningen inte fungerar?

Testa att ladda om sidan, logga ut och in igen, eller använd en annan webbläsare.

Steg 4: Ladda upp data till Snowflake



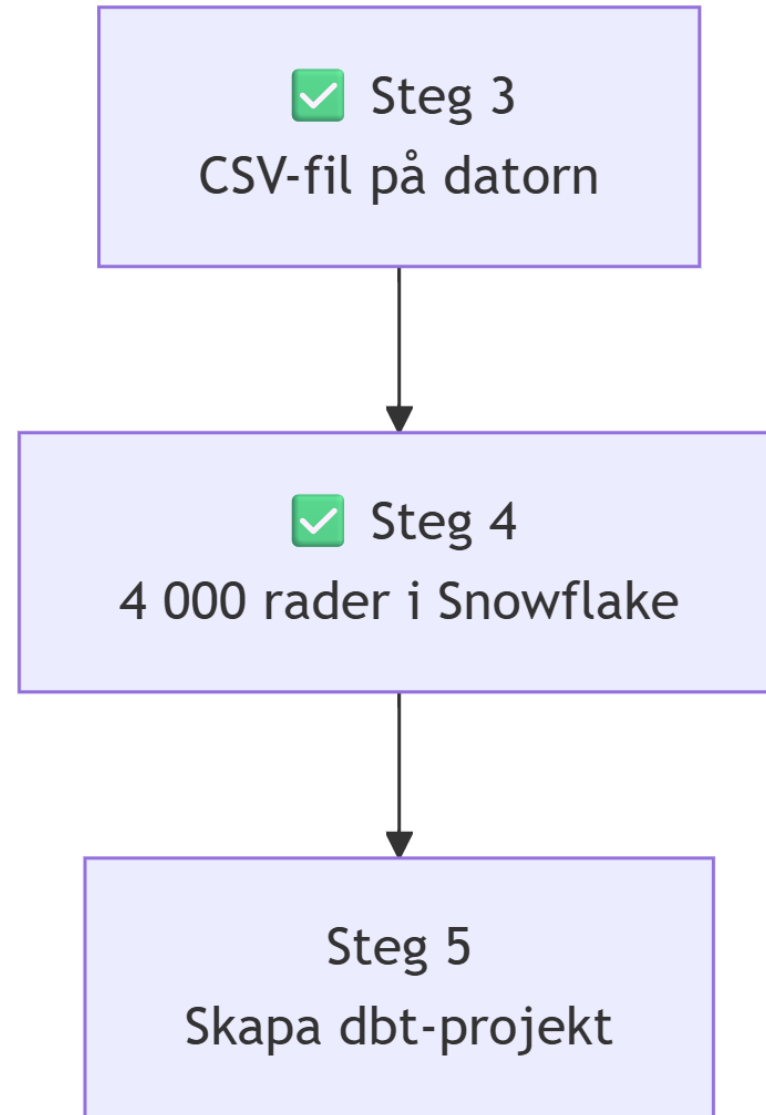
Ladda upp data till Snowflake

N	Detaljer	Värde / Instruktion
4.1	Logga in på Snowflake	Gå till din Snowflake-URL (t.ex. https://xxxxx.azure.snowflakecomputing.com)
4.2	Kontrollera roll	Högst upp till höger → välj ACCOUNTADMIN
4.3	Starta Compute-motorn	Manage → Compute → COMPUTE_WH → Klicka på namnet → "Resume" → Status ändras: Suspended → Starting → Started (grön)
4.4	Gå till "Upload local files"	Hem-sidan (klicka på Snowflake-loggan) → Under "Quick actions" → "Upload local files"
4.5	Ladda upp CSV-filen	Dra Medical_Internal_Attacks_Dataset.csv till rutan ELLER klicka på "or" och välj filen
4.6	Skapa databas	Rullgardin "Select or create a database" → Välj "Create new database" → Name: INSIDER_THREAT_DB → Comment: (lämna tomt) → klicka "Done"
4.7	Skapa tabell	Rullgardin "Select or create a table" → Välj "Create new table" → Table name: RAW_ACCESS_LOGS
4.8	Kontrollera kolumner	"Edit Schema" – visa 15 kolumner, kontrollera att attack_type finns sist
4.9	Klicka "Load"	Blå knapp längst ner → laddning tar 10-20 sekunder
4.10	Bekräfta laddning	Popup: 4,000 rows were successfully inserted
4.11	Verifiera med SQL	Gå till Worksheets → + Worksheet → kör: SELECT COUNT(*) FROM INSIDER_THREAT_DB.PUBLIC.RAW_ACCESS_LOGS; → Resultat: 4000
4.12	Visa data (valfritt)	Kör: SELECT * FROM INSIDER_THREAT_DB.PUBLIC.RAW_ACCESS_LOGS LIMIT 10; → Visar 10 rader med kolumner som duration, protocol_type, attack_type

Pro tips

Tips	Förklaring
Pausa Compute-motorn efteråt	Manage → Compute → COMPUTE_WH → "Suspend" – sparar dina krediter
Hitta tabellen i Catalog	Catalog → Databases → INSIDER_THREAT_DB → PUBLIC → RAW_ACCESS_LOGS
100 rader är normalt	Förhandsvisning visar bara 100 – alla 4000 finns där

Steg 4 är
klart



Så här stänger du av Compute-motorn (sparar dina krediter).

Din Compute-motor kostar krediter när den är igång. När du är klar för dagen pausar du den – då kostar den ingenting. Din data och databas sparas automatiskt.

Steg	Åtgärd	Var	Vad ser du?
1	Gå till Compute	Vänstermenyn → Manage → Compute	En lista med dina warehouses visas
2	Välj rätt warehouse	Leta efter raden COMPUTE_WH i tabellen	Status står "Started" (grön text)
3	Öppna detaljsidan	Klicka på namnet COMPUTE_WH (blå länk)	En ny sida med information visas
4	Klicka på "Suspend"	Uppe till höger – blå knappen " Suspend "	Knappen finns bredvid "Drop"
5	Bekräfta	Popup-fönster visas (välj "Suspend" eller "OK")	Vänta 5 sekunder
6	Kontrollera status	Status ändras från "Started" → " Suspended "	Textfärgen blir grå (inte grön längre)

Så här ser det ut före och efter:

Före (igång)	Efter (pausad)
Status: Started (grön)	Status: Suspended (grå)
Kostar krediter 	Kostar inget 
Knappen heter "Suspend"	Knappen heter "Resume"

Viktigt att veta

FRÅGA

SVAR

Försvinner min data när jag stänger av?

 Nej! Datan finns kvar i databasen INSIDER_THREAT_DB

Försvinner min databas?

 Nej – databasen är gratis och sparas automatiskt

Måste jag göra detta varje dag?

 Ja – för att spara dina gratiskrediter

Hur startar jag imorgon?

Gå till Compute → COMPUTE_WH → klicka "**Resume**"

Steg 5: Skapa dbt-projekt

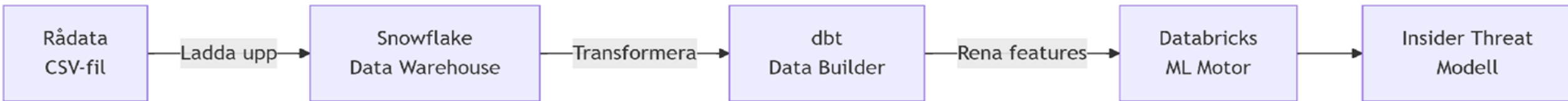


Du skapar ett dbt-projekt som ska transformera din rådata i Snowflake. dbt fungerar som ett lager mellan din rådata och din ML-modell – det skapar rena, strukturerade tabeller som Databricks kan använda.

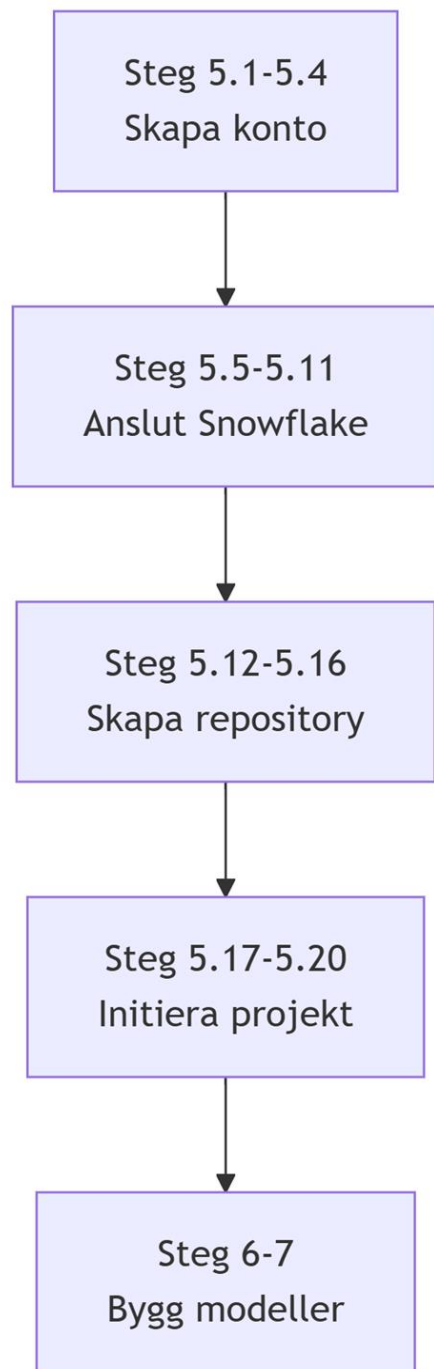


I den här guiden använder du **dbt Cloud** (gratisversionen via webbläsaren), vilket innebär att du inte behöver installera något lokalt. Allt görs i molnet precis som Snowflake och Databricks .

Steg 5: Skapa dbt-projekt



VERKTYG	VAD ÄR DET?	VARFÖR BEHÖVER DU DET?	VAD HÄNDER UTAN DET?
Snowflake	Ett molnbaserat data warehouse (databas i molnet)	Lagrar alla 4 000 rader av rådata på ett säkert och skalbart sätt	Du måste hantera data lokalt på din dator – vilket blir långsamt och opålitligt
dbt	Ett transformationsverktyg som rensar och strukturerar data med SQL	Gör din rådata ren och förberedd för maskininlärning – skapar features som "antal misslyckade inloggningar"	Din ML-modell får direkt tillgång till rå, ostrukturerad data – vilket ger dåliga eller felaktiga förutsägelser
Databricks	En plattform för maskininlärning och big data-analys	Bygger själva ML-modellen som upptäcker insiderhot baserat på dina rena features	Du kan inte träna någon modell alls – projektet stannar



Vad gör du i Steg 5?

- I Steg 5 skapar du ett **dbt-projekt** och kopplar ihop det med Snowflake. Detta är brobyggaren mellan din rådata (Snowflake) och din ML-modell (Databricks). Utan dbt skulle din ML-modell inte förstå datan.

N	Åtgärd	Exakt instruktion	Varför behövs detta?	Vad händer om du hoppar över?
5.1	Gå till dbt Cloud	Öppna https://www.getdbt.com/signup i din webbläsare	Du måste ha ett konto för att använda dbt	Du kan inte transformera din data – projektet stannar
5.2	Skapa konto	Fyll i: Namn, E-post, Lösenord (minst 9 tecken, stor bokstav, siffra, specialtecken). Klicka "Create my account"	dbt Cloud är webbaserat – inget att installera	Du måste installera dbt lokalt (mycket svårare)
5.3	Verifiera e-post	Gå till din e-postinkorg → klicka på länken från dbt Labs	dbt måste bekräfta att du är en riktig person	Kontot förblir inaktivt – du kan inte logga in
5.4	Svara på frågor	Välj "I am new to dbt" → "I'm learning dbt to advance my career" → klicka "Complete"	dbt anpassar din upplevelse som nybörjare	Du får avancerade tips som förvirrar dig
5.5	Skapa projekt	Klicka på "New Project" → skriv insider_threat_project → klicka "Continue"	Varje projekt är en separat arbetsyta för din kod	All din kod blandas ihop med andras projekt
5.6	Välj Snowflake	Klicka på "Snowflake" under "Connection settings"	Din data ligger i Snowflake – dbt måste prata med den	dbt kan inte hämta din data från Snowflake
5.7	Fyll i konto	Account t ex: jnqaoob-to29... (från din Snowflake-URL)	dbt behöver veta exakt var din Snowflake finns	Anslutningen misslyckas – "Server not found"
5.8	Fyll i databas	Database: INSIDER_THREAT_DB	dbt måste veta vilken databas din data ligger i	dbt letar på fel ställe – hittar inte din tabell
5.9	Fyll i warehouse	Warehouse: COMPUTE_WH	dbt behöver en motor för att köra SQL-frågor	Inga frågor kan köras – transformationer fungerar inte

5.10	Ange användarnamn	Username t ex: ARSENIY	dbt måste logga in på Snowflake som dig	Åtkomst nekad – "Access denied"
5.11	Ange lösenord	Password: Ditt Snowflake-lösenord	dbt måste autentisera sig mot Snowflake	Åtkomst nekad – "Invalid password"
5.12	Testa anslutning	Klicka "Test connection" → vänta på "Success"	Bekräftar att alla uppgifter är korrekta	Du får först reda på fel när du försöker köra dbt
5.13	Spara anslutning	Klicka "Save" → "Set up a repository"	Sparar dina Snowflake-uppgifter så att dbt kommer ihåg dem	Du måste skriva in dem varje gång
5.14	Välj repository-typ	Klicka på "Managed"	dbt skapar ett Git-repository åt dig automatiskt	Du måste skapa ett GitHub-konto (extra jobb)
5.15	Namnge repository	Skriv insider_threat_dbt_repo → klicka "Set up a repository"	Detta är var din dbt-kod sparas (versionshantering)	Du kan inte spara dina ändringar – risk att förlora arbete
5.16	Vänta på bekräftelse	Du ser "Success – Your project is ready"	Repositoryt är skapat och kopplat till ditt projekt	Projektskapandet är inte klart – du kan inte gå vidare
5.17	Öppna Studio	Klicka "Start developing in the Studio"	Studio är dbt:s utvecklingsmiljö (IDE) där du skriver SQL	Du kan inte skapa dina transformations-modeller
5.18	Initiera projekt	Klicka "Initialize dbt project" (uppe till höger)	Skapar alla nödvändiga mappar och filer (models/, dbt_project.yml, etc.)	Du har ingen mappstruktur – du vet inte var du ska skriva din kod
5.19	Vänta på filer	Du ser: dbt_project.yml, models/, seeds/, tests/, macros/, snapshots/, .gitignore, README.md	Bekräftar att projektet är korrekt initierat	Projektet är tomt – inget att arbeta med
5.20	Gör första commit	Klicka på "Version Control" → skriv "initial commit" → klicka "Commit and sync"	Sparar en första version av ditt projekt (som "spara" i Word)	Alla ändringar du gör är osparade – du kan förlora dem

Checklista

– Steg 5 är klart när:

Kontroll	 Klart?
Du har ett dbt Cloud-konto (gratis)	☐
dbt är ansluten till Snowflake (grönt "Success")	☐
Du har skapat ett managed repository	☐
Du har klickat "Initialize dbt project"	☐
Du ser mapparna models/, seeds/, tests/ i vänster sidofält	☐
Filen dbt_project.yml finns	☐
Du har gjort din första commit	☐

Så här ser det ut i dbt Cloud IDE:

Element	Var	Vad du ser
Mappstruktur	Vänstra sidofältet	models/ (här skapar du SQL- modeller), seeds/ (för CSV- filer), tests/ (för att testa)
dbt_project.yml	Roten av projektet	Huvudkonfigurationsfilen – titta men ändra inget än
SQL-editor	Mitten av skärmen	Här skriver du SELECT ... FROM ... för att transformera data
Versionskontr oll	Flik till vänster	Här ser du ändrade filer och gör commits
Kommando- rad	Längst ner	Här skriver du dbt run för att köra dina transformationer

Pro tips

Tips	Förklaring
Gör commits ofta	Efter varje ändring – klicka "Commit and sync". Det är som "Save" i Word.
dbt Cloud vs dbt Core	Du använder Cloud (webbaserat) – inget att installera. Core kräver terminal och är svårare.
Felmeddelanden är normalt	När du initierar projektet kan du se "Cannot open file" – det försvinner när projektet är klart.
Pausa Snowflake Compute	När du är klar för dagen – gå till Snowflake → Compute → COMPUTE_WH → "Suspend" (sparar krediter)
Kolla din trial-period	Du har 13 dagar kvar på dbt Starter trial – det räcker för hela projektet.

dbt är HELT gratis för dig – du behöver INTE göra något när trial-perioden tar slut

Fråga

Svar

Måste jag betala?

✗ Nej! Developer plan är **helt gratis** för en ensam utvecklare

Måste jag registrera nytt konto?

✗ Nej! Du behåller samma konto – det byter bara plan automatiskt

Måste jag fylla i betalkort?

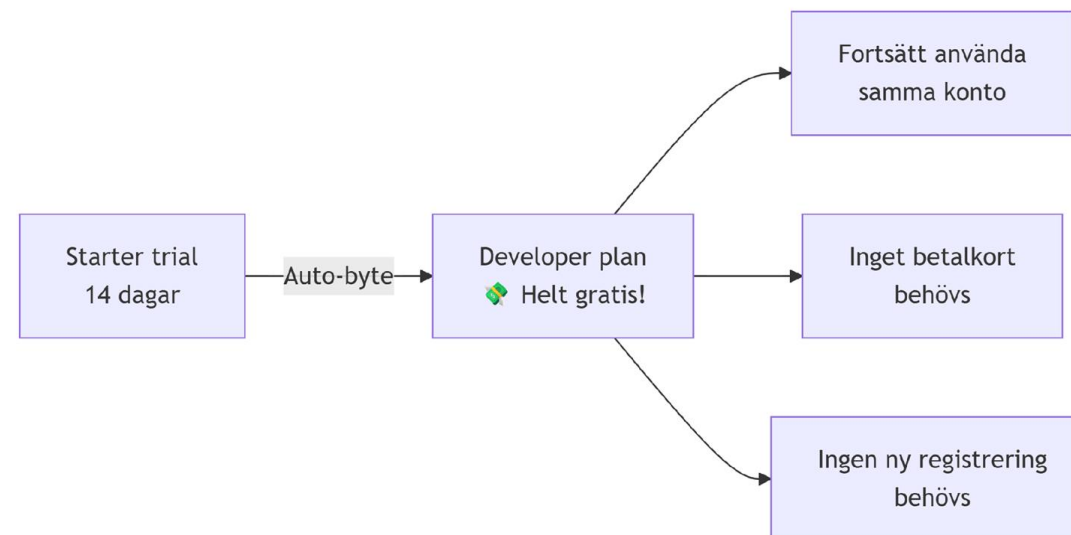
✗ Nej! Developer plan kräver inget betalkort

Förlorar jag mitt arbete?

✗ Nej! Alla dina projekt, modeller och inställningar sparas

Kan jag fortsätta

✓ Ja! Du kan använda dbt



Skillnad mellan planerna (för dig som ensam användare):

För vår projekt: 3 000 modeller per månad är långt mer än du behöver! Du kommer bara att skapa 3-5 modeller totalt.

Funktion	Starter (trial)	Developer (gratis)
Kostnad	Gratis i 14 dagar	Gratis för alltid
Antal användare	Upp till 5	1 (du)
Projekt	Obegränsat	1 projekt
IDE (Studio)	✓ Ja	✓ Ja
dbt run	✓ Ja	✓ Ja
Job scheduling	✓ Ja	✓ Ja
Modeller per månad	15 000	3 000



Vanliga frågor

Fråga	Svar
Måste jag betala för dbt?	Nej, du använder gratis trial-perioden som räcker för projektet.
Kan jag använda GitHub istället för Managed?	Ja, men Managed är enklast för nybörjare – inget extra konto behövs.
Vad är en "model" i dbt?	En model är en .sql-fil med en SELECT-fråga som skapar en ny tabell eller vy. I Steg 6 skapar du flera sådana.
Varför ser jag ett felmeddelande om dbt_project.yml?	Det är normalt innan du initierat projektet. Efter "Initialize dbt project" försvinner det.
Vad är nästa steg?	Steg 6 – skapa dbt-modeller för feature engineering (t.ex. login_attempts.sql, file_access.sql).

Sammanfattning – så här är det med alla dina verktyg:

Verktyg	Gratisversion	Behöver du göra något efter trial?
Snowflake	30-dagars trial	✓ Ja – skapa nytt konto (samma eller ny e-post) när 30 dagar är slut
dbt	Developer plan – gratis för alltid	✗ Nej – fortsätter automatiskt gratis
Databricks	Community Edition – gratis för alltid	✗ Nej – ingen tidsgräns

När Steg 5 är klart

✅ Steg 4
Data i Snowflake
4 000 rader

✅ Steg 5
dbt-projekt klart
Anslutet till Snowflake

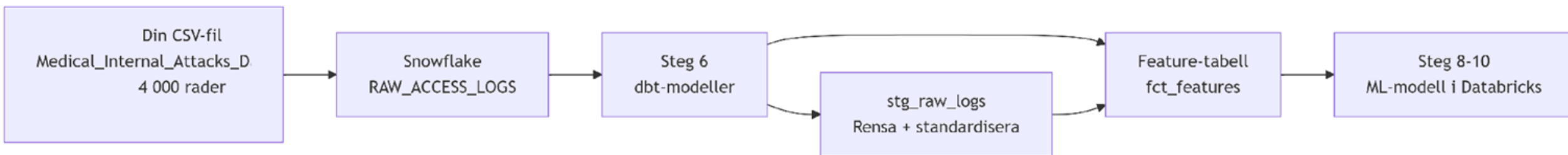
Steg 6
Skapa dbt-modeller
Feature engineering

Steg 7
Kör dbt-transformationer
dbt run

Steg 8-10
ML-modell i Databricks
Insider threat detection

Steg 6: Skapa dbt-modeller för feature engineering

- I Steg 6 skapar du **dbt-modeller** – SQL-filer som transformerar din rådata till **features** (egenskaper) som din ML-modell kan använda för att upptäcka insiderhot.
- **Enkelt förklarat:** Du gör om rådata (som är svår att förstå) till tydliga mönster som en maskin kan lära sig från.

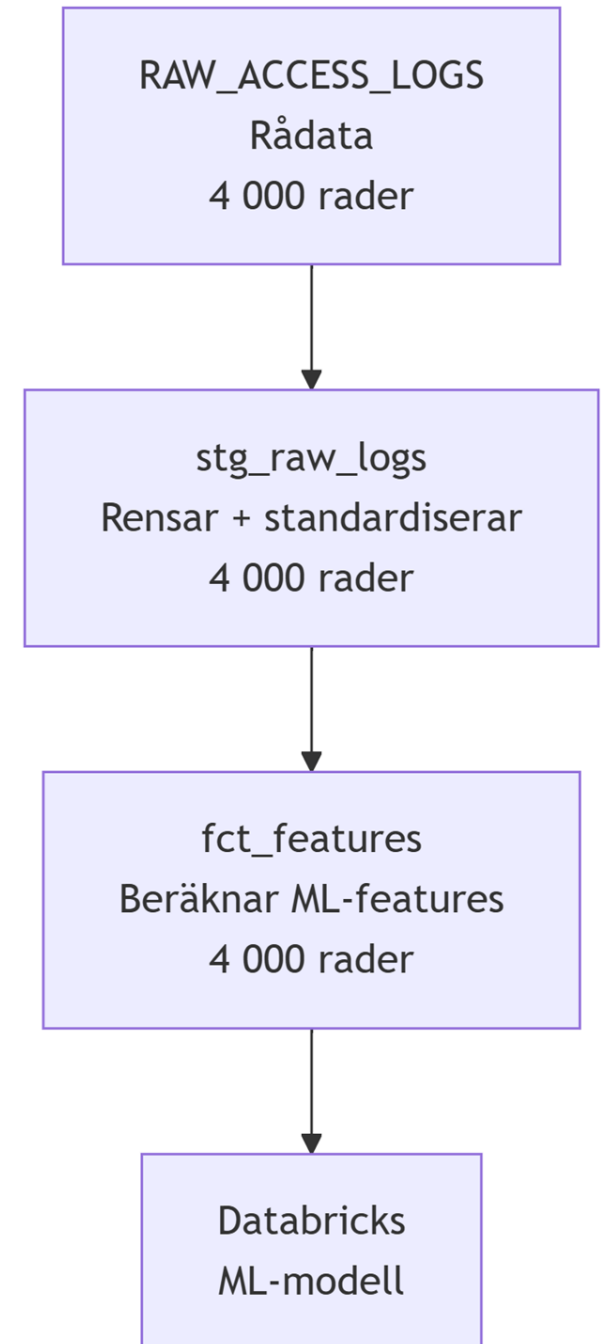


Vilka modeller ska du skapa?

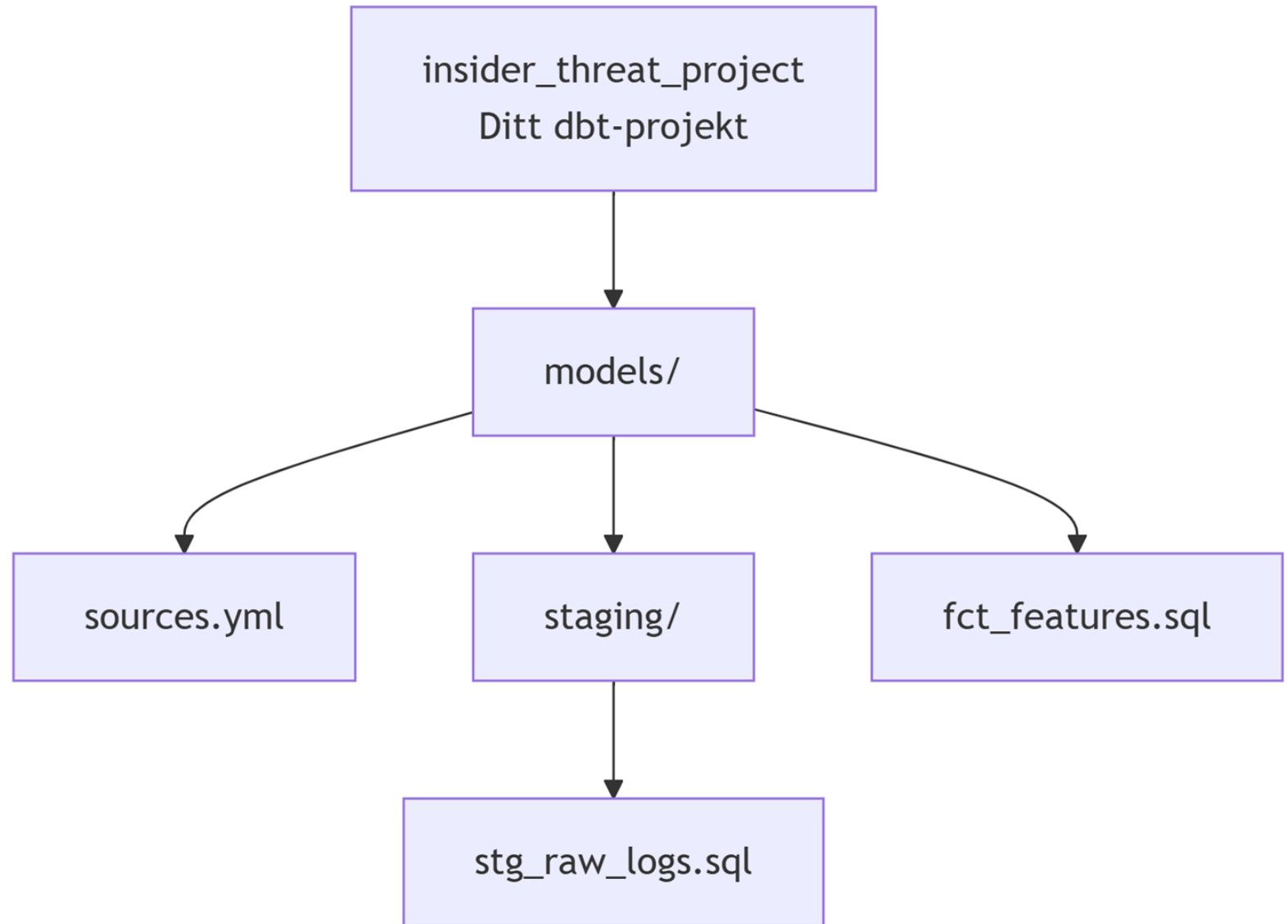
MODELL-FIL	VAD DEN GÖR	VARFÖR BEHÖVS DEN?	ANTAL RADER EFTER
sources.yml	Berättar för dbt var rådatan finns i Snowflake	Utan denna vet dbt inte var din data är	—
stg_raw_logs.sql	Rensar och standardiserar rådatan (EN staging-modell)	Skapar en återanvändbar, ren version av datan	4 000
fct_features.sql	Skapar färdiga features för ML	Huvudtabellen som din ML-modell använder	4 000

Varför bara EN staging- modell?

- ☒ Undviker CROSS JOIN (som ger miljarder rader)
- ☒ Enklare för nybörjare att förstå
- ☒ Fortfarande industri-standard för mindre projekt
- ☒ Snabbare att köra och felsöka



Mappstruktur efter Steg 6



Steg 6: Komplett tabell

N	ÅTGÄRD	VAR	EXAKT INSTRUKTION	VARFÖR BEHÖVS DETTA?
6.1	Öppna dbt Cloud IDE	dbt Cloud	Klicka på "Studio" i vänstermenyn	Här skapar du dina modeller
6.2	Ta bort exempelmappen	Vänster sidofält	Högerklicka på models/example → "Delete"	Exempelfilerna är bara för demo
6.3	Skapa staging-mapp	Högerklicka på models → "Create folder"	Skriv: staging → klicka "Create"	Håller dina modeller organiserade
6.4	Skapa sources.yml	Högerklicka på models → "Create file"	Skriv: sources.yml	Berättar för dbt var rådatan finns
6.5	Klistra in YAML-kod	I sources.yml	Klistra in koden från tabellen "Kod för sources.yml"	dbt får veta databas, schema och tabellnamn
6.6	Spara sources.yml	Ctrl+S (Windows) / Cmd+S (Mac)	Klicka "Save"	Sparar din konfiguration
6.7	Skapa stg_raw_logs.sql	Högerklicka på staging → "Create file"	Skriv: stg_raw_logs.sql	Första och enda staging-modellen
6.8	Klistra in SQL-kod	I den nya filen	Klistra in koden från tabellen "Kod för stg_raw_logs.sql"	Rensar och standardiserar rådatan
6.9	Spara filen	Ctrl+S / Cmd+S	Klicka "Save"	Sparar dina ändringar
6.10	Skapa fct_features.sql	Högerklicka på models → "Create file"	Skriv: fct_features.sql	Huvudmodell för ML-features
6.11	Klistra in SQL-kod	I den nya filen	Klistra in koden från tabellen "Kod för fct_features.sql"	Skapar färdiga features för ML
6.12	Spara filen	Ctrl+S / Cmd+S	Klicka "Save"	Sparar dina ändringar
6.13	Kontrollera att alla filer är sparade	Vänster sidofält	Du ser: sources.yml, staging/stg_raw_logs.sql, fct_features.sql	Alla filer är på plats
6.14	Gör commit (valfritt)	Version Control → "Commit and sync"	Skriv: added staging and fct models	Sparar en version av ditt arbete

Fil 1: models/sources.yml

```
version: 2
```

```
sources:
```

- name: snowflake
description: "Snowflake-databasen där vår rådata finns"
database: INSIDER_THREAT_DB
schema: PUBLIC
tables:
 - name: RAW_ACCESS_LOGS
description: "Rådata från Medical Internal Attack Detection dataset (4 000 rader)"
columns:
 - name: attack_type
description: "Målvariabel - typ av insiderhot"
 - name: num_failed_logins
description: "Antal misslyckade inloggningsförsök"
 - name: flag
description: "Statusflagga (SF=normal, REJ=avvisad, S0=fel)"

Fil 2: models/staging/stg_raw_logs.sql

```
-- =====
-- STAGING MODELL: Rensar och standardiserar rådata
-- =====
-- Syfte: Förbereda rådata för ML-modellen
-- Källa: INSIDER_THREAT_DB.PUBLIC.RAW_ACCESS_LOGS
-- Antal rader: 4 000 (oförändrat)
-- =====

SELECT
  -- SKAPA UNIK IDENTIFIERARE
  CONCAT(protocol_type, '_', service) AS user_session_id,

  -- ORIGINALKOLUMNER (behålls oförändrade)
  attack_type,           -- Målvariabel för ML
  num_failed_logins,     -- Misslyckade inloggningar
  root_shell,           -- Root-åtkomst (1=ja, 0=nej)
  su_attempted,         -- Privilegieeskaleringsförsök
  num_access_files,     -- Åtkomst till patientjournaler
  num_file_creations,   -- Skapade filer
  src_bytes,            -- Skickad data
  dst_bytes,            -- Mottagen data
  duration,             -- Sessionlängd i sekunder
  same_srv_rate,        -- Andel samma tjänst
  diff_srv_rate,        -- Andel olika tjänster
  flag,                 -- Statusflagga
  logged_in,            -- Inloggningsstatus

  -- BERÄKNADE FÄLT
  CASE
    WHEN flag IN ('REJ', 'S0') THEN 'high_risk'
    WHEN flag = 'SF' THEN 'normal'
    ELSE 'unknown'
  END AS risk_category,

  -- METADATA
  CURRENT_TIMESTAMP() AS processed_at

FROM {{ source('snowflake', 'RAW_ACCESS_LOGS') }}
```

Fil 3: models/fct_features.sql

```
-- =====
-- HUVUDMODELL FÖR ML: Features för Insider Threat Detection
-- =====
-- Syfte: Skapa färdig feature-tabell för maskininlärning
-- Källa: stg_raw_logs (staging-modellen ovan)
-- Antal rader: 4 000 (oförändrat)
-- =====

SELECT
  -- IDENTIFIERARE
  user_session_id,

  -- MÅLVARIABEL
  attack_type,

  -- ORIGINALFEATURES
  num_failed_logins,
  root_shell,
  su_attempted,
  num_access_files,
  num_file_creations,
  src_bytes,
  dst_bytes,
  duration,
  same_srv_rate,
  diff_srv_rate,
  flag,
  logged_in,
  risk_category,

  -- BERÄKNADE FEATURES (för ML-modellen)
  (num_failed_logins + root_shell + su_attempted) AS total_privilege_attempts,
  (src_bytes + dst_bytes) AS total_bytes_transferred,

  -- NUMERISK RISKSCORE (för ML-algoritmer)
  CASE
    WHEN flag IN ('REJ', 'S0') THEN 2
    ELSE 0
  END AS flag_risk_score

FROM {{ ref('stg_raw_logs') }}
```

Checklista

– Steg 6 är klart när:

KONTROLL	☒ KLART?
Du har tagit bort models/example-mappen	☐
Mappen staging/ finns under models/	☐
Filen sources.yml finns i models/	☐
Filen stg_raw_logs.sql finns i models/staging/	☐
Filen fct_features.sql finns i models/	☐
Alla filer är sparade (Ctrl+S efter varje)	☐
(Valfritt) Du har gjort commit	☐

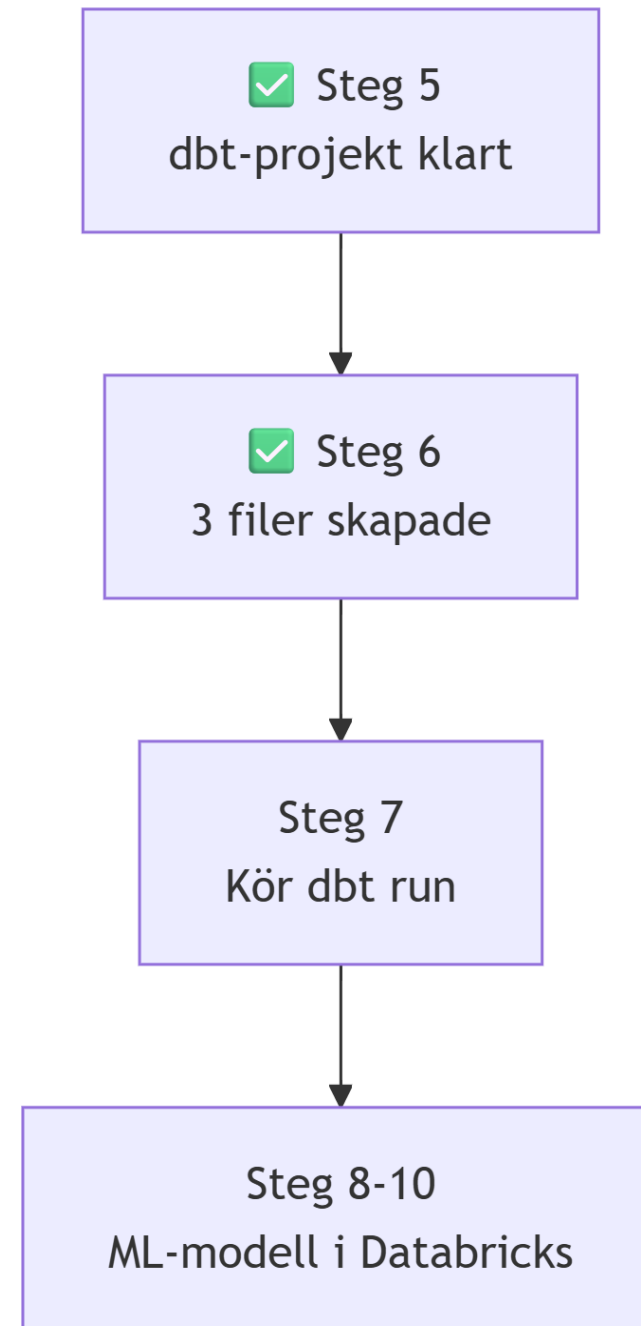
Pro tips

TIPS	FÖRKLARING
Börja med EN staging-modell	Det räcker för ett projekt av denna storlek. Lägg till fler när du behöver återanvända logik.
source vs ref	source används för rådata som redan finns i Snowflake. ref används för dina egna dbt-modeller.
Inga JOINS i staging	Staging-modeller ska bara rensa data – inga JOINS eller summeringar. Det gör man i fct_features.
Spara ofta	Efter varje SQL-fil – spara! (Ctrl+S). dbt sparar inte automatiskt.
Testa COUNT(*) efter dbt run	Kontrollera att antalet rader är 4 000 – inte 78 miljarder!

Vanliga frågor

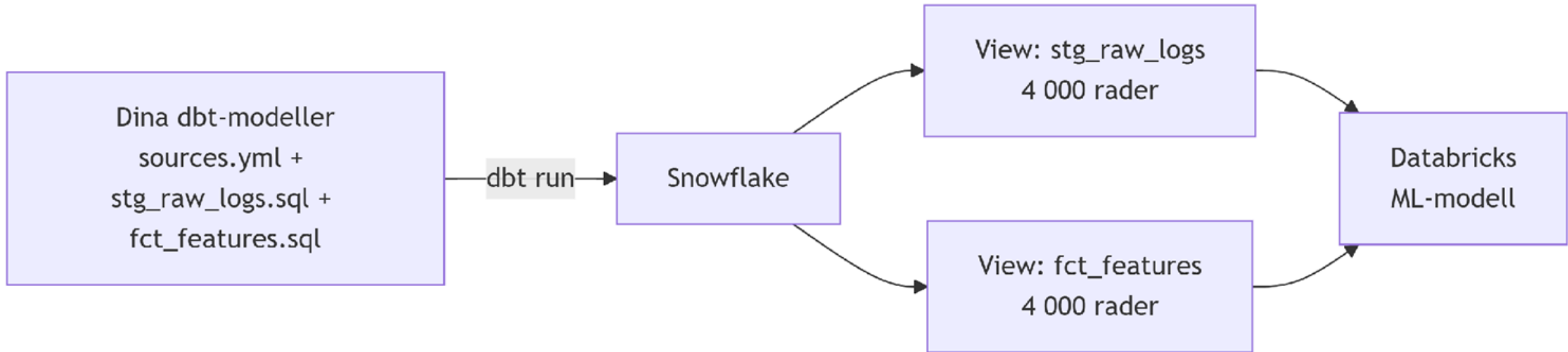
Fråga	Svar
Varför bara EN staging-modell?	Du har bara EN rådatatabell. En staging-modell räcker för att rensa och standardisera.
Vad är skillnaden på stg_ och fct_?	stg = staging (rensar data). fct = fact (beräknar features för ML).
Behöver jag fler staging-modeller?	Nej, för ditt projekt räcker en. I större projekt kan man ha fler.
Vad är nästa steg?	Steg 7 – köra dbt run för att skapa tabellerna i Snowflake!

När Steg 6
är klart –
gå vidare
till Steg 7



Steg 7: Kör dbt-transformationer

- I Steg 7 kör du dina dbt-modeller så att de skapar tabeller (views) i Snowflake. dbt översätter din SQL-kod till riktiga databastabeller som du sedan kan använda i Databricks för maskininlärning.
- **Enkelt förklarat:** Du trycker på "start" – dbt bygger dina features som riktiga tabeller i molnet.



Steg 7: Komplett tabell

N	Åtgärd	Var	Exakt instruktion	Varför behövs detta?
7.1	Öppna kommandoraden	dbt Cloud IDE (längst ner i fönstret)	Klicka på fliken " Commands " längst ner	Här skriver du dbt-kommandon
7.2	Kontrollera att du är i rätt projekt	Kommandoraden	Du ser projektnamnet i terminalen	Bekräftar att du är på rätt plats
7.3	Skriv dbt run-kommandot	Kommandoraden	Skriv: dbt run och tryck Enter	Bygger ALLA dina modeller i Snowflake
7.4	Vänta medan dbt arbetar	Kommandoraden	Du ser meddelanden som "Running...", "Writing..."	dbt översätter SQL till databaskommandon
7.5	Kontrollera resultatet	Kommandoraden	Leta efter " Completed successfully " (grön text)	Bekräftar att alla modeller byggdes
7.6	Kontrollera antalet rader (valfritt)	Snowflake webbplats	Logga in på Snowflake → Worksheets → kör SQL	Verifierar att du har 4 000 rader (inte miljarder!)

Kommandon du kommer att använda

Kommando	Vad det gör	När använder du det?	Förväntat resultat
dbt run	Bygger ALLA dina modeller	Första gången du kör – skapar alla tabeller/views	Completed successfully, 2 models built
dbt run --select fct_features	Bygger BARA huvudmodellen	När du bara vill uppdatera feature-tabellen	Completed successfully, 1 model built
dbt run --select staging	Bygger BARA staging-modellen	När du bara vill uppdatera den rensade datan	Completed successfully, 1 model built
dbt ls	Listar alla dina modeller	För att se vilka modeller som finns	stg_raw_logs, fct_features

Så här ser det ut i kommandoraden (exempel på utskrift)

- `$ dbt run`

Running with `dbt=1.8.0`

Found `2` models, `0` tests, `0` snapshots, `0` analyses, `0` macros

Concurrency: `6` threads

`1 of 2` START sql view model dbt_ab.stg_raw_logs [RUN]

`1 of 2` OK created sql view model dbt_ab.stg_raw_logs [SUCCESS `0.9s`]

`2 of 2` START sql view model dbt_ab.fct_features [RUN]

`2 of 2` OK created sql view model dbt_ab.fct_features [SUCCESS `0.34s`]

Completed successfully

Done. `PASS=2` `WARN=0` `ERROR=0` `SKIP=0` `TOTAL=2`

Vad betyder utskriften?

Del av utskriften	Betydelse
Found 2 models	dbt hittade dina 2 SQL-filer (stg_raw_logs och fct_features)
1 of 2 START	dbt börjar bygga första modellen (stg_raw_logs)
OK created sql view model	Modellen skapades som en "view" (virtuell tabell)
SUCCESS 0.9s	Modellen byggdes på 0.9 sekunder
PASS=2 WARN=0 ERROR=0	Alla 2 modeller lyckades, inga fel
Completed successfully	Hela körningen var lyckad

Kontrollera i Snowflake (valfritt steg 7.6)

N	Åtgärd	Var	SQL-kommando
1	Logga in på Snowflake	Webbläsaren	Gå till din Snowflake-URL
2	Starta Compute-motorn	Manage → Compute → COMPUTE_WH	Klicka " Resume " (om den är pausad)
3	Öppna en Worksheet	Worksheets → "+ Worksheet"	Skriv SQL-kod
4	Byt till rätt schema	Worksheet	USE SCHEMA INSIDER_THREAT_DB.DBT_AB;
5	Kontrollera antalet rader	Worksheet	SELECT COUNT(*) FROM FCT_FEATURES;
6	Visa de första raderna	Worksheet	SELECT * FROM FCT_FEATURES LIMIT 5;

Förväntade resultat i Snowflake:

SQL-kommando

Förväntat resultat

```
SELECT COUNT(*) FROM  
FCT_FEATURES;
```

4 000 (inte 78 miljarder!)

```
SELECT * FROM  
FCT_FEATURES LIMIT 5;
```

5 rader med kolumner
som ATTACK_TYPE, NUM_FAILED_LOGINS, etc.

Checklista – Steg 7 är klart när:



Du har öppnat kommandoraden (fiken "Commands")



Du har kört dbt run



Du ser "Completed successfully" (grön text)



dbt run visar PASS=2 (2 modeller lyckades)



(Valfritt) SELECT COUNT(*) i Snowflake visar 4 000



Vanliga fel och lösningar

FELMEDDELANDE	VAD DET BETYDER	LÖSNING
Database error: Table 'RAW_ACCESS_LOGS' does not exist	dbt hittar inte din rådata	Kontrollera sources.yml – rätt databas/schema/tabellnamn?
Compilation error: source not found	dbt hittar inte källan	Kontrollera att sources.yml finns i models/-mappen
Warehouse 'COMPUTE_WH' does not exist	Snowflake-motorn är inte startad	Logga in på Snowflake → Compute → starta COMPUTE_WH
Access denied	Inloggningsproblem	Kontrollera Snowflake-uppgifterna i dbt-anslutningen
Syntax error	Fel i SQL-koden	Kontrollera att du klistrat in koden exakt som i Steg 6

Pro tips

Tips	Förklaring
Starta Snowflake Compute-motorn först	Innan du kör dbt run – gå till Snowflake och starta COMPUTE_WH (Resume). Annars misslyckas dbt.
Kör dbt run igen om det failar	dbt är idempotent – du kan köra det flera gånger. Det förstör inte data.
dbt run bygger bara nya/ändrade modeller	Andra gången du kör går det mycket snabbare.
Kolla loggarna vid fel	dbt skriver detaljerade felmeddelanden – scrolla upp i kommandoraden.
Pausa Compute-motorn efteråt	När du är klar med Steg 7 – gå till Snowflake och pausa COMPUTE_WH (Suspend) för att spara krediter.
Fliken "Commands" längst ner	Om du inte ser kommandoraden – klicka på fliken "Commands" längst NER i IDE-fönstret.

Vad händer i Snowflake efter dbt run?

Tabell/View	Typ	Antal rader	Vad den innehåller
STG_RAW_LOGS	View	4 000	Rensad och standardiserad rådata
FCT_FEATURES	View	4 000	Alla features + attack_type – redo för ML!

Vanliga frågor

Fråga

Svar

Måste jag köra dbt run varje gång jag ändrar en modell?

Ja! dbt bygger inte om automatiskt. Kör dbt run efter varje ändring.

Kan jag köra dbt run flera gånger?

Ja, det är helt säkert. dbt ersätter bara gamla views med nya.

Varför blev båda modellerna views?

Standardkonfigurationen i dbt skapar views. Det är bra för utveckling.

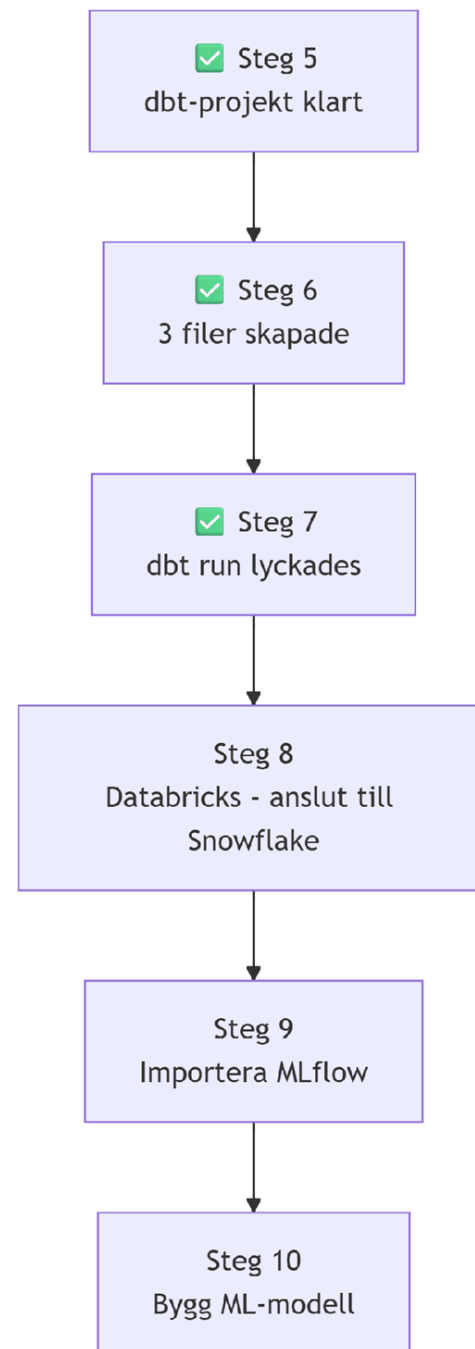
Jag ser inte fliken "Commands" – vad gör jag?

Titta längst NER i IDE-fönstret. Klicka på "Commands". Annars tryck Ctrl+J (Windows) eller Cmd+J (Mac).

Vad är nästa steg?

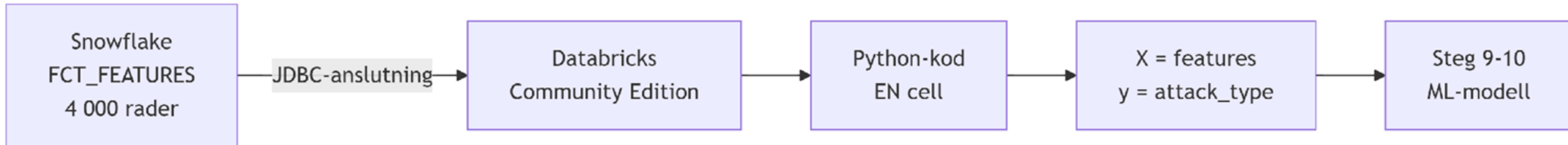
Steg 8 – Ansluta Databricks till Snowflake och börja bygga ML-modellen!

Din väg framåt



Steg 8: Anslut Databricks till Snowflake

- I Steg 8 ansluter du Databricks till Snowflake och hämtar din färdiga feature-tabell `FCT_FEATURES`. Detta är sista steget innan du bygger din ML-modell.
- **Enkelt förklarat:** Du hämtar datan från Snowflake till Databricks där ML-modellen byggs.

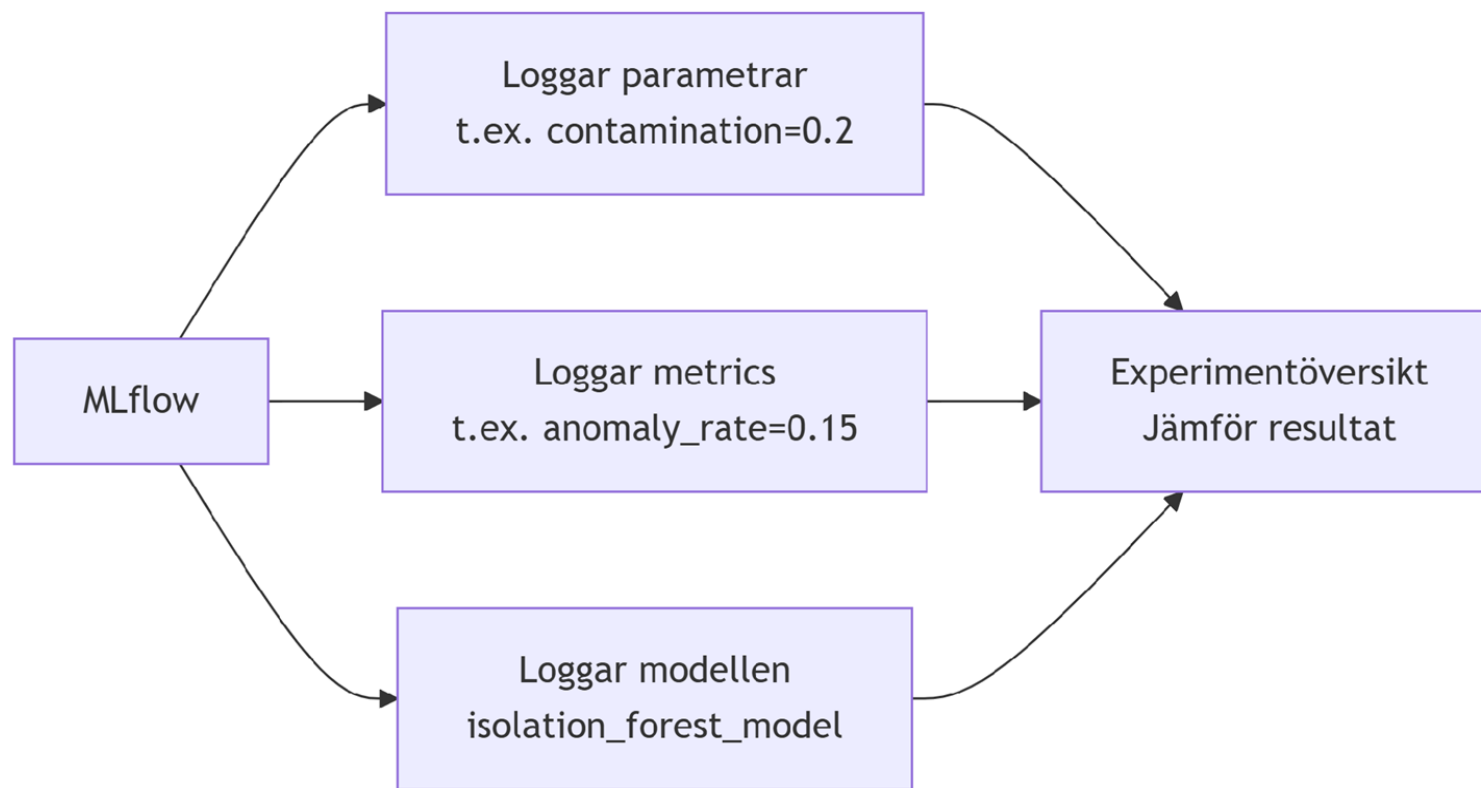


Steg 8: Komplett tabell (EN cell)

N	Åtgärd	Kodrad	Förklaring
8.1	Installera bibliotek	<code>%pip install snowflake-connector-python pandas scikit-learn -q</code>	Behövs för att prata med Snowflake och för ML
8.2	Importerera bibliotek	<code>import snowflake.connector as sf import pandas as pd from getpass import getpass</code>	Verktygen för anslutning, datahantering och lösenord
8.3	Anslut till Snowflake	<code>conn = sf.connect(account='...', user='...', password=getpass(), ...)</code>	Skapar en bro mellan Databricks och Snowflake
8.4	Ange lösenord	Skrivs i terminalen (syns inte)	Autentisering mot Snowflake
8.5	Kontrollera antal rader	<code>SELECT COUNT(*) FROM FCT_FEATURES</code>	Verifierar att du har 4 000 rader (inte miljarder!)
8.6	Hämta all data	<code>df = pd.read_sql(query, conn)</code>	Laddar datan till Databricks som en Pandas DataFrame
8.7	Stäng anslutningen	<code>conn.close()</code>	Frigör resurser (undviker timeout)
8.8	Förbered för ML	<code>y = df['ATTACK_TYPE'] X = df[numerical_cols]</code>	Separerar målvariabel från features
8.9	Visa resultat	<code>df.head()</code>	Kontrollerar att datan ser rätt ut

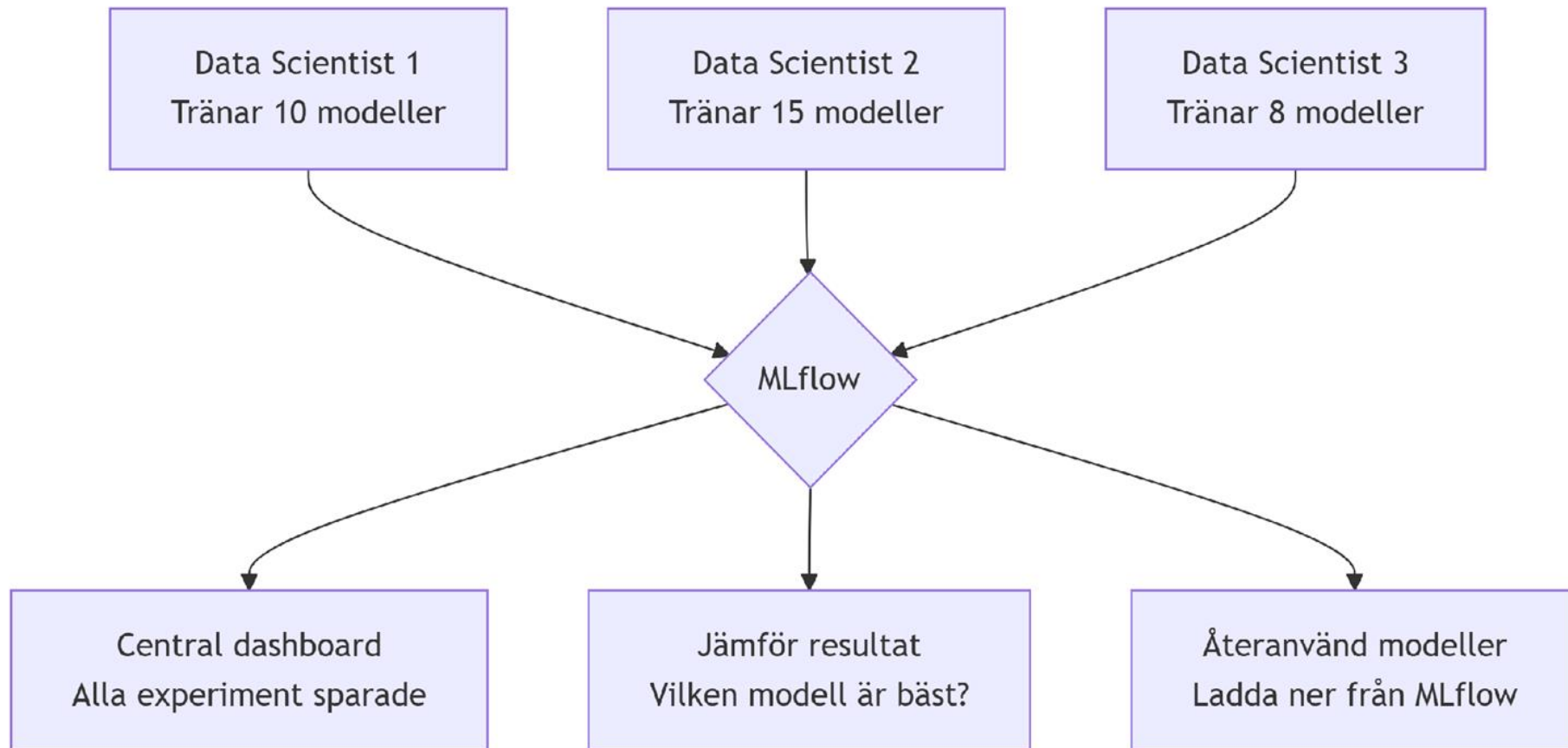
Steg 9: Importera MLflow i Databricks

- I Steg 9 importerar du **MLflow** – ett verktyg som sparar alla dina ML-experiment. Det gör att du kan jämföra olika modeller, spara vilka parametrar du använde, och se resultaten i en snygg översikt.
- **Enkelt förklarat:** MLflow är som en **dagbok** för din ML-modell – den sparar allt du gör så att du kan gå tillbaka och se vad som fungerade bäst.



Varför MLflow används i professionell produktion

- **MLflow är standardverktyget för att spåra ML-experiment i företag.** Utan MLflow blir det kaos när flera personer tränar flera modeller samtidigt.



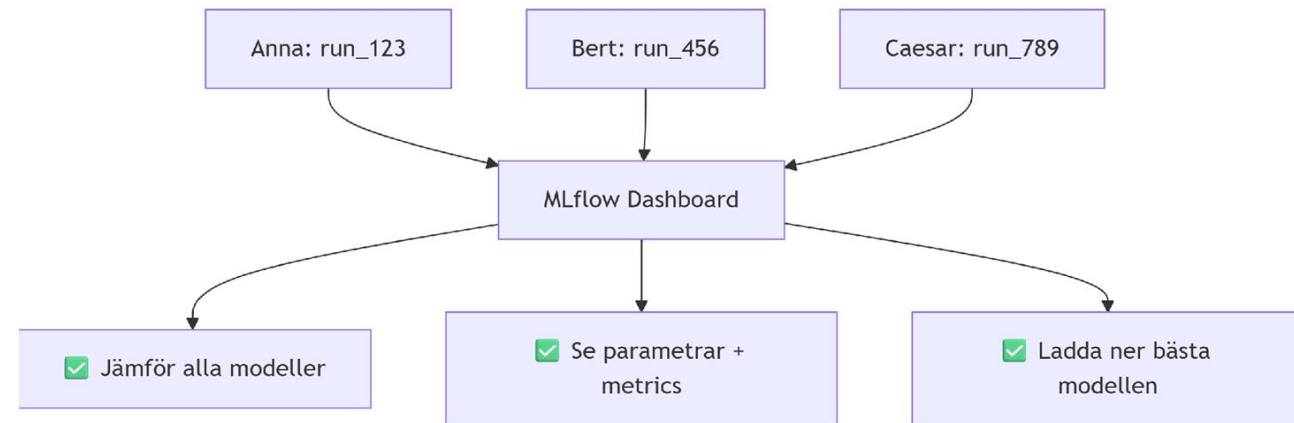
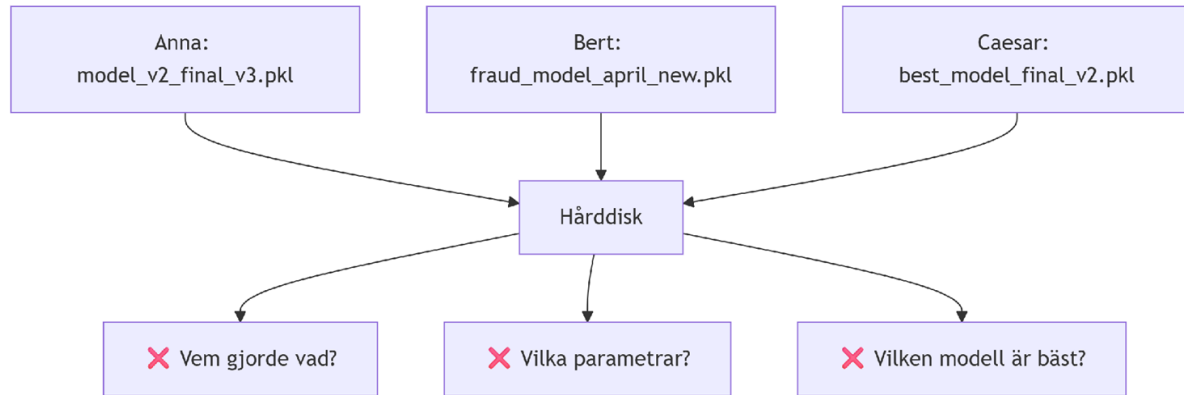
Verkliga exempel från företag

Företag	Hur de använder MLflow	Problem de löser
Spotify	Spårar miljarder musikrekommendationer	"Vilken modell gav bäst spellistor förra veckan?"
Airbnb	Loggar alla pris-prediktioner	"Varför funkade modellen bättre i Paris än i New York?"
Uber	Spårar ETA-modeller (ankomsttid)	"Behöver vi träna om modellen varje dag?"
Netflix	Loggar rekommendationsmodeller	"Vilken parametrar gav högst tittartid?"

Steg 9: Kompletta tabell (EN cell)

N	Åtgärd	Kod	Förklaring
9.1	Importera MLflow	<code>import mlflow</code>	Huvudbiblioteket för att logga experiment
9.2	Importera tracking-modul	<code>from mlflow.tracking import MlflowClient</code>	För att hantera experiment och körningar
9.3	Importera sklearn (för att spara modeller)	<code>import mlflow.sklearn</code>	För att spara scikit-learn-modeller
9.4	Skapa ett experiment	<code>mlflow.set_experiment("/Users/din_email/insider_threat")</code>	Skapar en mapp för alla dina körningar
9.5	Verifiera att MLflow fungerar	<code>print(f"MLflow version: {mlflow.__version__}")</code>	Kontrollerar att allt är installerat

Verkligt scenario: Ett ML-team på 5 personer



**Utan
MLflow
(kaos):**

**Med MLflow
(ordning):**

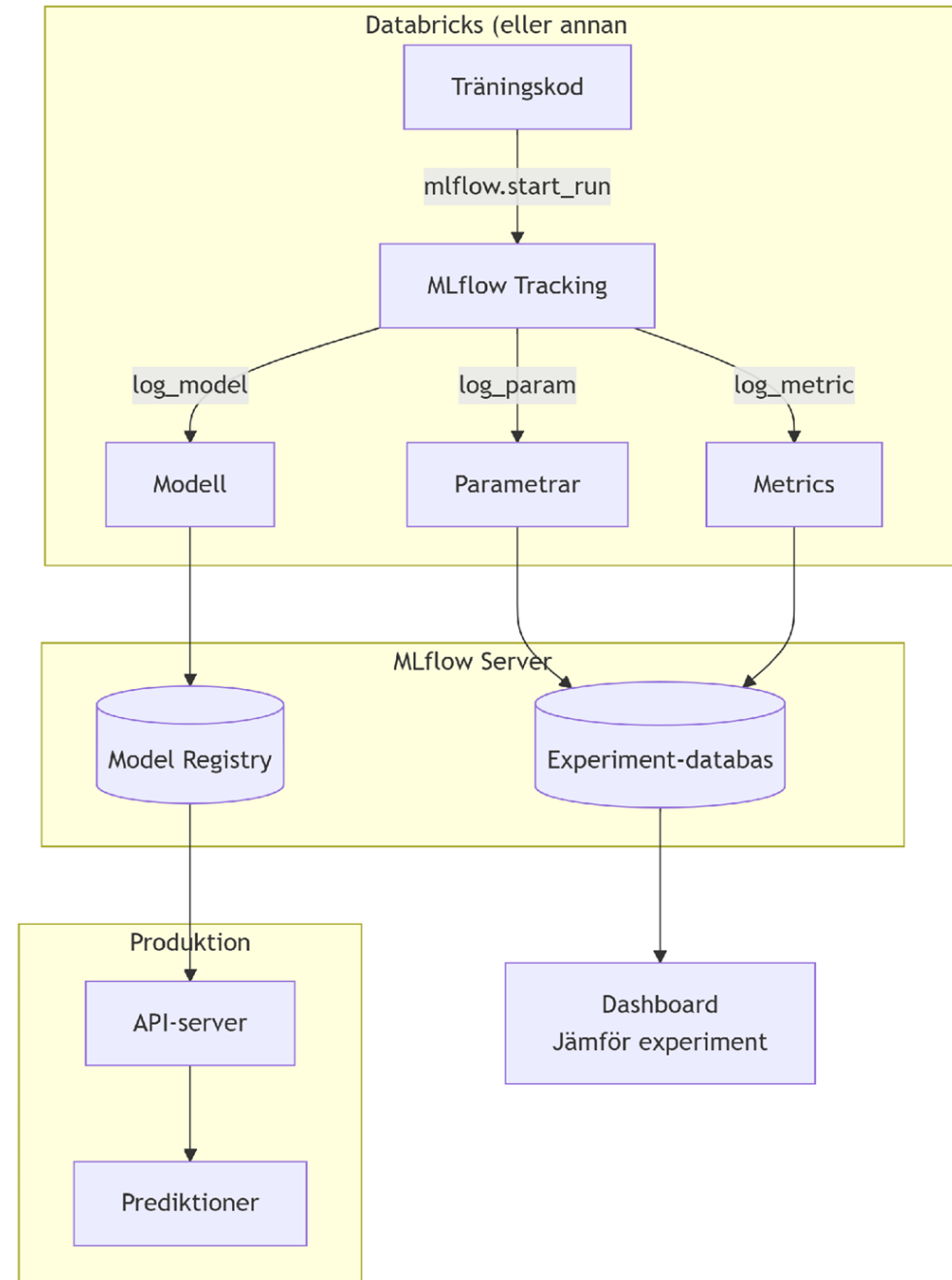
Utan vs Med MLflow

Aspekt	Utan MLflow	Med MLflow
Spara parametrar	Skriv manuellt i Excel eller textfil	✓ Automatiskt – <code>mlflow.log_param()</code>
Spara metrics	Kopiera från terminalen	✓ Automatiskt – <code>mlflow.log_metric()</code>
Spara modellen	.pkl fil på en hårddisk	✓ I MLflow som "artifact"
Jämföra modeller	Öppna 10 olika filer	✓ En dashboard med alla körningar
Samarbete	Maila filer fram och tillbaka	✓ Alla ser samma experiment
Reproducerbarhet	"Vilka parametrar använde jag?"	✓ Allt sparas – kör exakt samma igen
Tid för att hitta bästa modell	2-3 timmar manuellt	5 minuter i MLflow UI

Utan vs Med MLflow

Aspekt	Utan MLflow	Med MLflow
Spara parametrar	Skriv manuellt i Excel eller textfil	✓ Automatiskt – <code>mlflow.log_param()</code>
Spara metrics	Kopiera från terminalen	✓ Automatiskt – <code>mlflow.log_metric()</code>
Spara modellen	.pkl fil på en hårddisk	✓ I MLflow som "artifact"
Jämföra modeller	Öppna 10 olika filer	✓ En dashboard med alla körningar
Samarbete	Maila filer fram och tillbaka	✓ Alla ser samma experiment
Reproducerbarhet	"Vilka parametrar använde jag?"	✓ Allt sparas – kör exakt samma igen
Tid för att hitta bästa modell	2-3 timmar manuellt	5 minuter i MLflow UI

MLflow- arkitektur i produktion



Varför just MLflow (och inte annat)?

Verktyg	Fördelar	Nackdelar	Används av
MLflow	✓ Gratis, öppen källa, integrerat med Databricks	–	Airbnb, Uber, Netflix
TensorBoard	✓ Bra för deep learning	✗ Fungerar bara för TensorFlow	Deep learning-team
Weights & Biases	✓ Snygg dashboard	✗ Kostar pengar (enterprise)	Startup med budget
Excel/Google Sheets	✓ Gratis	✗ Manuellt, glömmer saker, inget samarbete	Används fortfarande! (men dåligt)

Steg 10: Bygg ML-modell (Random Forest)

- I Steg 10 bygger du en **Random Forest-modell** som upptäcker insiderhot. Random Forest är en **övervakad** ML-modell – den lär sig från exempel på attacker och normala beteenden.
- **Enkelt förklarat:** Modellen lär sig mönster från 4 000 exempel och kan sedan avgöra om en ny användare beter sig misstänkt.



Steg 10: Översikt

Steg	Åtgärd	Vad görs?	Resultat
10.1	Skapa binär etikett	Attack (1) vs Normal (0)	87.6% attack, 12.3% normal
10.2	Dela upp data	80% träning, 20% test	3 200 / 800 rader
10.3	Skala features	Normalisera värden	Alla kolumner på skala 0-1
10.4	Träna Isolation Forest (jämförelse)	Oövervakad modell	F1-score: 0.658
10.5	Träna Random Forest	Övervakad modell	F1-score: 0.979
10.6	Jämför modeller	Precision, Recall, F1	RF vinner
10.7	Analysera viktigaste features	Vad påverkar mest?	TOTAL_PRIVILEGE_ATTEMPTS 46%
10.8	Logga till MLflow	Spara experiment	Gjort!

Isolation Forest vs Random Forest

MÄTVÄRDE	ISOLATION FOREST	RANDOM FOREST	VARFÖR RF ÄR BÄTTRE
Precision	87.3%	99.1%	Färre falska larm
Recall	52.8%	96.7%	Upptäcker nästan alla attacker
F1-score	0.658	0.979	Mycket bättre balans
Falska positiva	54 st	6 st	Mycket färre falska larm
Missade attacker	331 st	23 st	Upptäcker nästan alla

Confusion Matrix (förklaras)

		Förutsagd	
		Normal	Attack
← 6 falska positiva	Normal	93	6
	Verklig Attack	23	678
← 23 missade attacker			

Cell	Antal	Betydelse	Bra/dåligt?
Övre vänster	93	✓ Normala som inte flaggades	Bra!
Övre höger	6	✗ Normala som flaggades (falskt alarm)	Mycket bra!
Nedre vänster	23	✗ Attacker som missades	Bra (bara 3%)
Nedre höger	678	✓ Attacker som upptäcktes	Utmärkt!

Viktigaste features (vad modellen lär sig)

Feature	Viktighet	Förklaring	Visualisering
TOTAL_PRIVILEGE_ATTEMPTS	46.2%	Summan av alla privilegieförsök	
NUM_FAILED_LOGINS	29.6%	Misslyckade inloggningar	
ROOT_SHELL	4.2%	Root-åtkomst	
SU_ATTEMPTED	3.1%	Privilegieeskalering	
SAME_SRV_RATE	2.2%	Återanvändning av samma tjänst	

Tolkning: Modellen har lärt sig att TOTAL_PRIVILEGE_ATTEMPTS (summan av misslyckade inloggningar + root + su_attempted) är den **starkaste indikatorn** på insiderhot – vilket är helt logiskt!

Slutgiltiga resultat

Mätvärde	Ditt resultat	Betydelse	Bedömning
Precision	99.1%	När modellen flaggar, har den rätt i 99 av 100 fall	★ Utmärkt
Recall	96.7%	Modellen hittar 97 av 100 attacker	★ Utmärkt
F1-score	0.979	Perfekt balans mellan precision och recall	★ Utmärkt
Falska positiva	6 st	Bara 6 normala beteenden flaggades	★ Mycket bra
Missade attacker	23 st	Bara 23 av 701 attacker missades	★ Mycket bra

Vad är detta multi-agent tillägg?

- Efter att din Random Forest-modell tränats (Steg 10), lägger du till **4 AI-agenter** som samarbetar för att analysera och förbättra resultatet.

Random Forest
99.1% precision

Agent 1
ML-analytiker

Agent 2
Säkerhetsexpert

Agent 3
Rapportskrivare

Agent 4
Kvalitetsgranskare

 Slutrapport
till ledningen

Utan LLM vs Med LLM (multi-agent)

Aspekt	Utan LLM (bara Random Forest)	Med LLM (multi-agent)
Vad du får	Siffror: 99.1% precision, 96.7% recall	Förståelse + rekommendationer
Kan chefen förstå?	✗ Nej – bara tekniska siffror	✓ Ja – rapport på svenska
Vet du vad du ska göra härnäst?	✗ Nej	✓ Ja – "samla in 500 nya exempel"
Varför fungerar modellen?	✗ Vet inte	✓ "TOTAL_PRIVILEGE_ATTEMPTS är viktigast"
Tid att fatta beslut	Timmar (analysera siffror själv)	Minuter (AI gör jobbet)

Vad varje agent gör (och varför)

Agent	Roll	Vad den gör	Exempel från din körning
1	ML-analytiker	Läser dina siffror (99.1% precision, 23 missade)	"STYRKA: Hög precision, SVAGHET: 23 missade"
2	Säkerhetsexpert	Tänker som en säkerhetschef	"RISK: Missade attacker kan leda till incidenter"
3	Rapportskrivare	Skriver så att chefen förstår	"Rapport till ledningen: 99.1% precision..."
4	Kvalitetsgranskare	Kortar ner och lägger till nästa steg	"Nästa steg: Samla in 500 nya exempel"

Resultat från DIN körning (steg för steg)

Steg	Vad hände	Resultat
Innan	Bara Random Forest siffror	99.1% precision, 96.7% recall, 23 missade
Agent 1	Analyserade siffrorna	"Hög precision men 23 missade attacker"
Agent 2	Tolkade säkerhetsrisken	"Risk: missade attacker kan vara allvarliga"
Agent 3	Skrev rapport till ledning	Professionell text på svenska
Agent 4	Förbättrade rapporten	Kortare + "Nästa steg: samla in 500 exempel"
Slutresultat	Åtgärdbar rekommendation	 Du vet EXAKT vad du ska göra härnäst!

Mono-agent vs Multi-agent (viktig skillnad)

Aspekt	Mono-agent (1 AI)	Multi-agent (4 AI)
Antal AI	1	4
Samarbete	 Nej – en AI gör allt	 Ja – specialiserade roller
Kvalitetskontroll	 Ingen	 Agent 4 granskar
Specialisering	 En AI måste kunna allt	 Var sin expertis
Resultatkvalitet	★ ★ OK	★ ★ ★ ★ ★ Mycket bättre
Används i produktion	Nej (för enkelt)	Ja (PGA TOUR, Workday, Accenture)

Varför LLM blir allt mer populärt (i produktion)

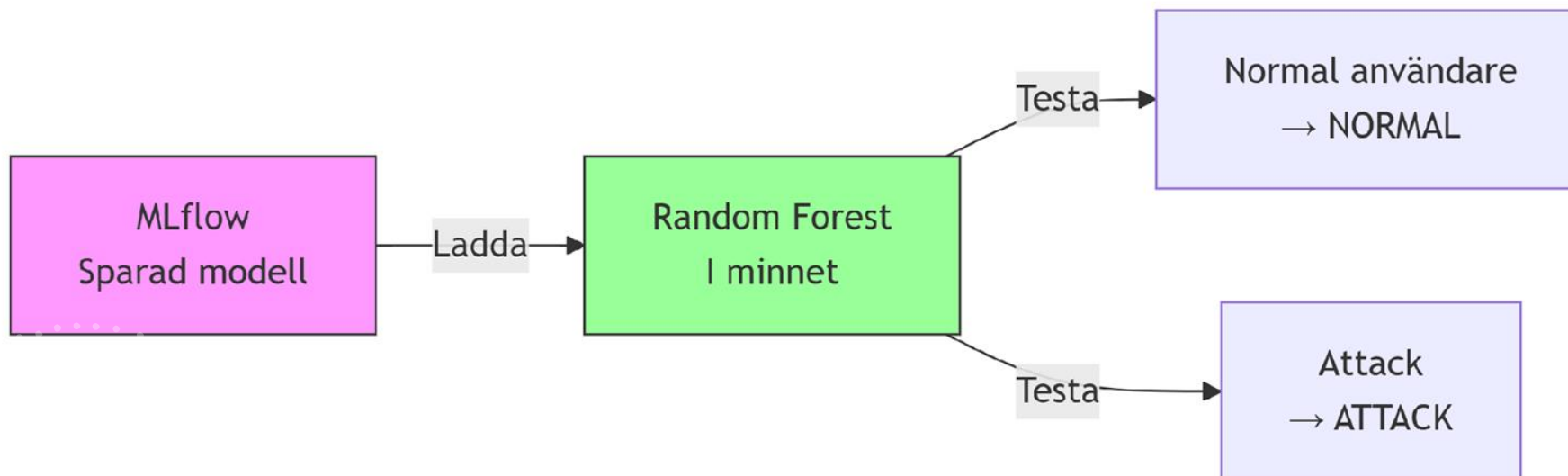
Anledning	Förklaring	Verkligt exempel
Förklarar ML-modeller	Random Forest ger bara siffror – LLM förklarar VARFÖR	Ditt projekt: "TOTAL_PRIVILEGE_ATTEMPT S är viktigast"
Automatiserar rapporter	Ingen vill läsa 100 sidor teknisk dokumentation	Agent 3 skrev rapport på 30 sekunder
Sparar tid	Från timmar till minuter	Du slapp analysera siffror själv
Gör AI begriplig	Chefer förstår inte F1-score, men förstår "99% precision"	Din slutrapport är läsbar för alla
Multi-agent samarbete	En AI kan ha fel – fyra AI som granskar varandra är säkrare	Agent 4 förbättrade Agent 3:s rapport

FAQ: Vanliga frågor och svar

#	Fråga	Kort svar	Detaljer
1	Behöver jag starta om alla celler varje morgon?	✗ Nej	Databricks sparar allt automatiskt i molnet. Variablerna (X, y, modell) finns kvar även efter att du stängt webbläsaren.
2	Vad händer med min API-nyckel när jag stänger webbläsaren?	✗ Försvinner	API-nyckeln finns bara i minnet. Nästa dag måste du ange den igen – det är säkrare så!
3	Sparas Random Forest-modellen automatiskt?	✓ Ja	Din kod i Steg 10/11 använder <code>mlflow.sklearn.log_model()</code> som sparar modellen i MLflow.
4	Sparas multi-agent systemet (LLM) automatiskt?	✗ Nej	LLM är ett API-anrop (Grok/Gemini/Groq). Ingen modell att spara – bara API-nyckel behövs nästa dag.
5	Kan jag ladda Random Forest från MLflow nästa dag?	✓ Ja	Använd <code>mlflow.sklearn.load_model()</code> med ditt run-id.
6	Kan jag använda multi-agent LLM nästa dag?	✓ Ja	Ange bara API-nyckeln igen – ingen modell att ladda.
7	Behöver jag spara modellen i separat fil också?	✗ Nej	MLflow räcker. Vill du vara extra säker kan du använda <code>joblib.dump()</code> som backup.
8	Vad är skillnaden mellan Random Forest och LLM?	 RF = tränad modell  LLM = API-anrop	RF sparas som en fil i MLflow. LLM är ett moln-API – du anropar det varje gång.

Steg 11: MLflow – Ladda och använd sparad modell

- I Steg 11 laddar du din sparade Random Forest-modell från MLflow och testar den på olika scenarier. Detta visar att modellen kan **användas i produktion** för att upptäcka insiderhot.
- **Enkelt förklarat:** Du hämtar din sparade modell från "modellbiblioteket" och testar att den fungerar på nya exempel.



Steg 11: Komplet t tabell

STEG	ÅTGÄRD	KOD	FÖRKLARING
11.1	Förstå MLflow	<code>print("MLflow är inbyggt...")</code>	System för att spåra ML-experiment
11.2	Hitta Run ID	Leta i Databricks → Experiment	Unikt ID för din modell
11.3	Ladda modell	<code>mlflow.sklearn.load_model()</code>	Hämtar modellen från molnet
11.4	Ladda scaler	<code>joblib.load(scaler_path)</code>	Hämtar skalningsobjektet
11.5	Visa metrics	<code>mlflow.get_run(RUN_ID)</code>	Visar precision, recall, f1
11.6	Testa modell	<code>loaded_model.predict()</code>	Gör förutsägelser på ny data

Varje student får ett unikt Run ID

Hitta ditt Run ID:

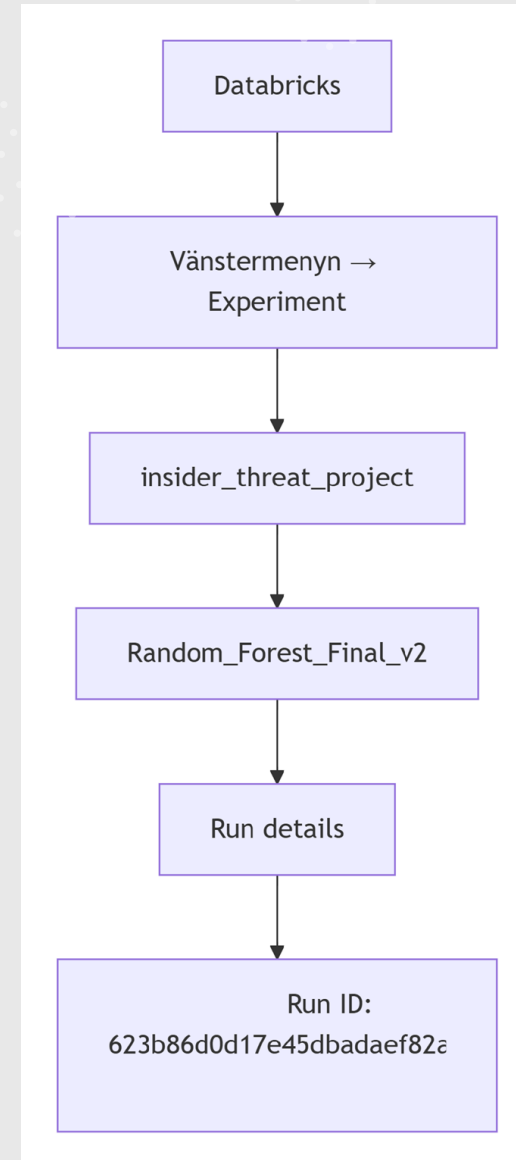
1. Gå till vänstermenyn → "Experiment"
2. Klicka på "insider_threat_project"
3. Klicka på din körning (t.ex. "Random_Forest_Final_v2")
4. Kopiera "Run ID" från "Run details"

FRÅGA	DITT SVAR
Var hittar man Run ID?	I Databricks → Experiment → din körning → Run details
Är det samma för alla?	Nej – varje gång man tränar en modell får man ett NYTT unikt Run ID
Vad gör man med det?	Man byter ut RUN_ID = "..." mot sitt eget ID
Behöver man ändra annat?	Nej – bara Run ID

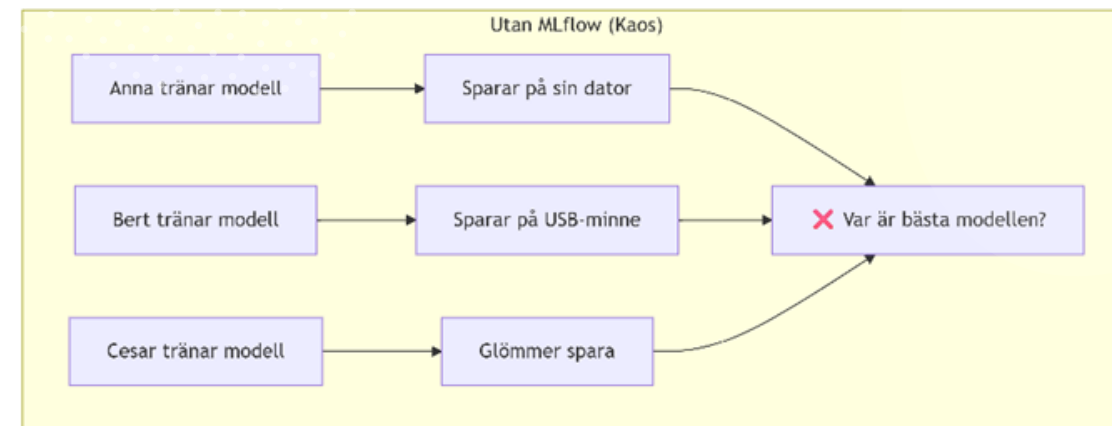
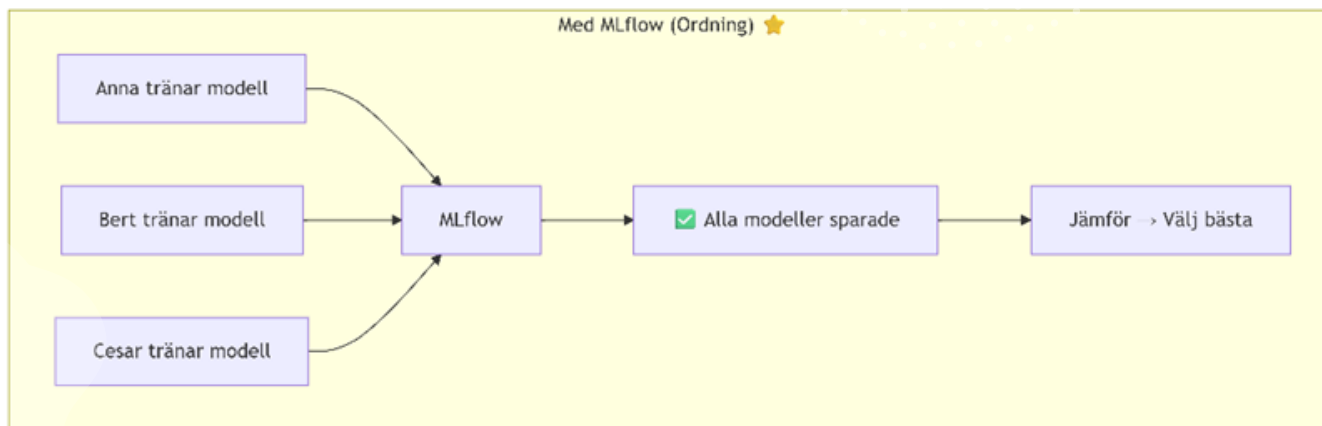
Var hittar du Run ID?

N Åtgärd

- 1 Gå till **vänstermenyn** i Databricks
- 2 Klicka på "**Experiment**"
- 3 Klicka på insider_threat_project
- 4 Klicka på körningen Random_Forest_Final_v2
- 5 I "**Run details**" ser du **Run ID**



Varför MLflow behövs



Verkligt företag

Problem

Spotify

100+ data scientists tränar rekommendationsmodeller

Uber

ETA-modeller tränas varje dag för varje stad

Airbnb

Pris-prediktioner för 7 miljoner annonser

Lösning med MLflow

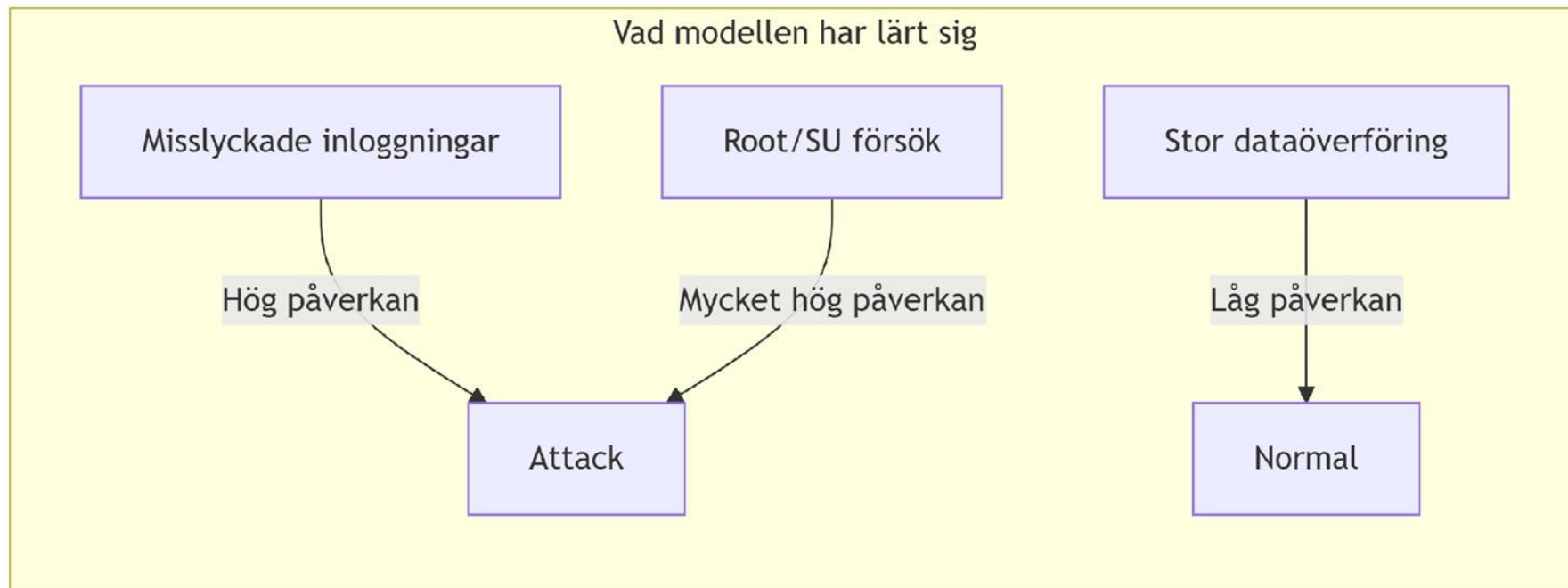
Alla modeller sparas centralt, bästa väljs

Automatisk spårning – vilken modell gav bäst ETA?

Jämför modeller – vilken gav högst intäkter?

Dina testresultat (vad modellen lärde sig)

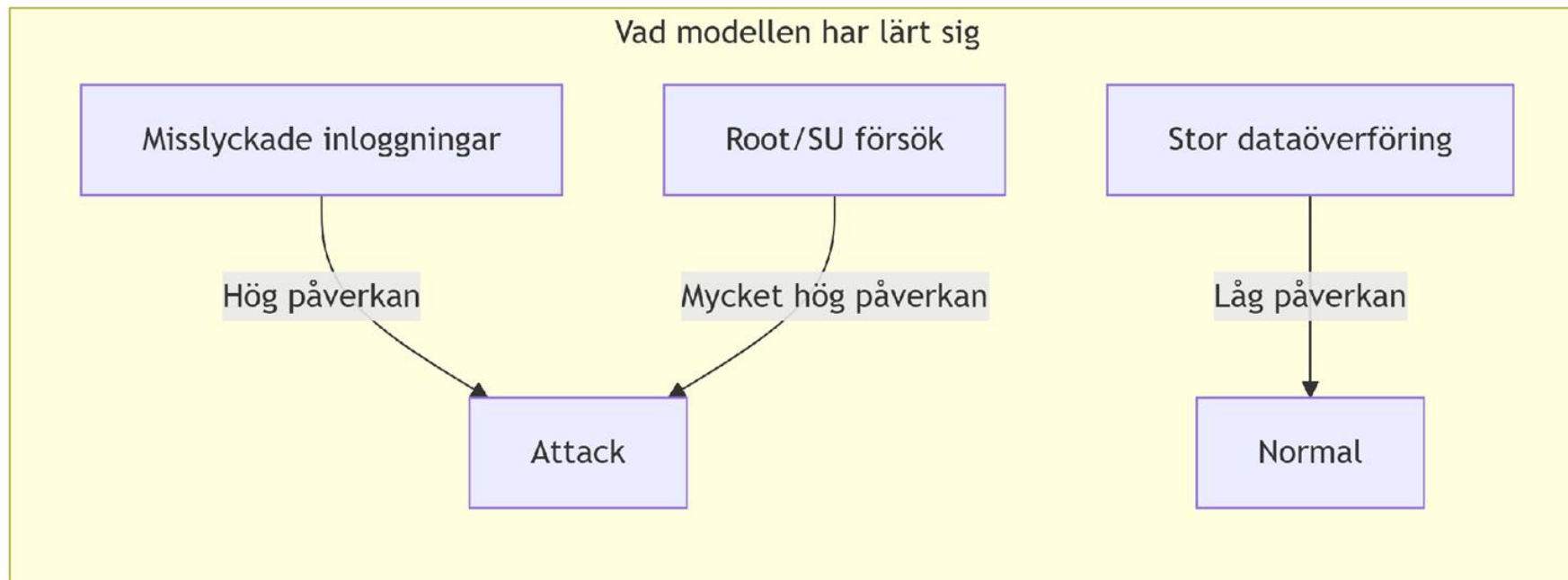
Test	Scenario	Modellens dom	Sannolikhet	Rätt/Fel
1	Normal användare	✓ NORMAL	94.4%	✓ Rätt
2	5 misslyckade inloggningar	⚠ ATTACK!	97.0%	✓ Rätt
3	Root + SU attempt	⚠ ATTACK!	100.0%	✓ Rätt



✓ Rätt

Dina testresultat (vad modellen läste sig)

Test	Scenario	Modellens dom	Sannolikhet	Rätt/Fel
1	Normal användare	✓ NORMAL	94.4%	✓ Rätt
2	5 misslyckade inloggningar	⚠ ATTACK!	97.0%	✓ Rätt
3	Root + SU attempt	⚠ ATTACK!	100.0%	✓ Rätt
4	Stor dataöverföring	✓ NORMAL	91.0%	✓ Rätt



Så här
skulle detta
användas i
riktig
produktion

STEG I		
PRODUKTION	VAD SOM HÄNDER	MLFLOWS ROLL
1	Ny data kommer in (användarbeteende)	–
2	Modellen laddas från MLflow	<code>mlflow.sklearn.load_model(</code> <code>)</code>
3	Förutsägelse görs	<code>model.predict()</code>
4	Resultat loggas tillbaka till MLflow	<code>mlflow.log_metric()</code>
5	Om modellen blir sämre – byt till äldre version	Model Registry

Varför MLflow är industri- standard

Utan MLflow

✗ Modeller sparas på olika datorer

✗ "Vilken modell var bäst?" – vet inte

✗ Kan inte återskapa resultat

✗ Ingen spårning för revisorer

✗ Svårt att byta modell i produktion

Med MLflow

✓ Alla modeller centralt

✓ Jämför metrics i dashboard

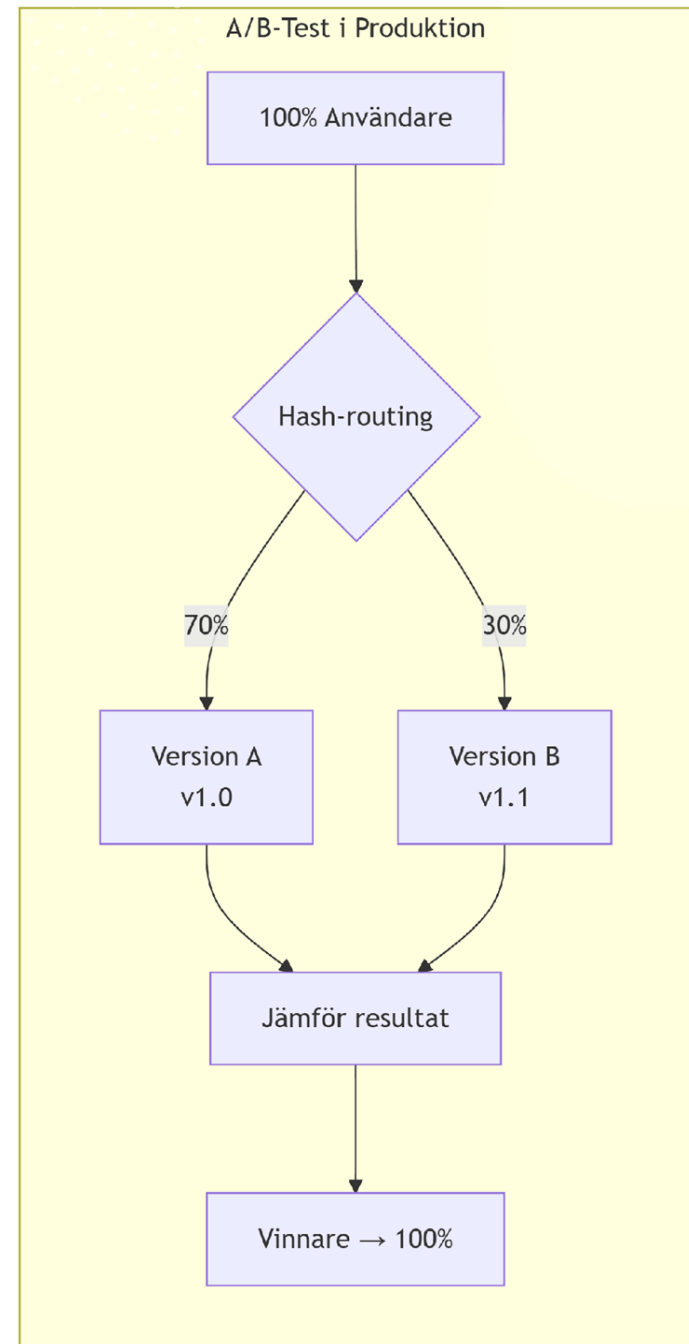
✓ Reproducerbarhet

✓ Full spårbarhet

✓ Model Registry med versioner

Steg 11.4: A/B-Testning – Jämför modellversioner

- Du A/B-testar **två versioner av samma Random Forest-modell** för att se vilken som presterar bäst i produktion. Detta är exakt hur Netflix, Spotify och Uber testar nya modellversioner innan de rullas ut till alla användare.



Steg 11.4: Komplett tabell

N	Åtgärd	Kod	Förklaring
11.4.1	Ladda Version A (v1.0)	mlflow.sklearn.load_model()	Din bästa modell från Steg 10
11.4.2	Träna Version B (v1.1)	RandomForestClassifier(n_estimators=200, max_depth=15)	Ny version med fler träd och djupare träd
11.4.3	Offline-jämförelse	precision_score(), recall_score(), f1_score()	Jämför på testdata innan produktion
11.4.4	Simulera A/B-test	Hash-baserad routing (70/30 trafikdelning)	70% användare → v1.0, 30% → v1.1
11.4.5	Analysera resultat	Jämför korrekthet i live-testet	Välj vinnare
11.4.6	Rekommendation	Baserat på resultat	Behåll v1.0 eller uppgradera till v1.1

Resultat från din A/B- testning. Offline- jämförelse (testdata)

Version	Parametrar	Precision	Recall	F1-score	Accuracy
v1.0	n_estimators=100, max_depth=10	99.1%	96.7%	0.979	96.4%
v1.1	n_estimators=200, max_depth=15	98.7%	97.0%	0.978	96.3%

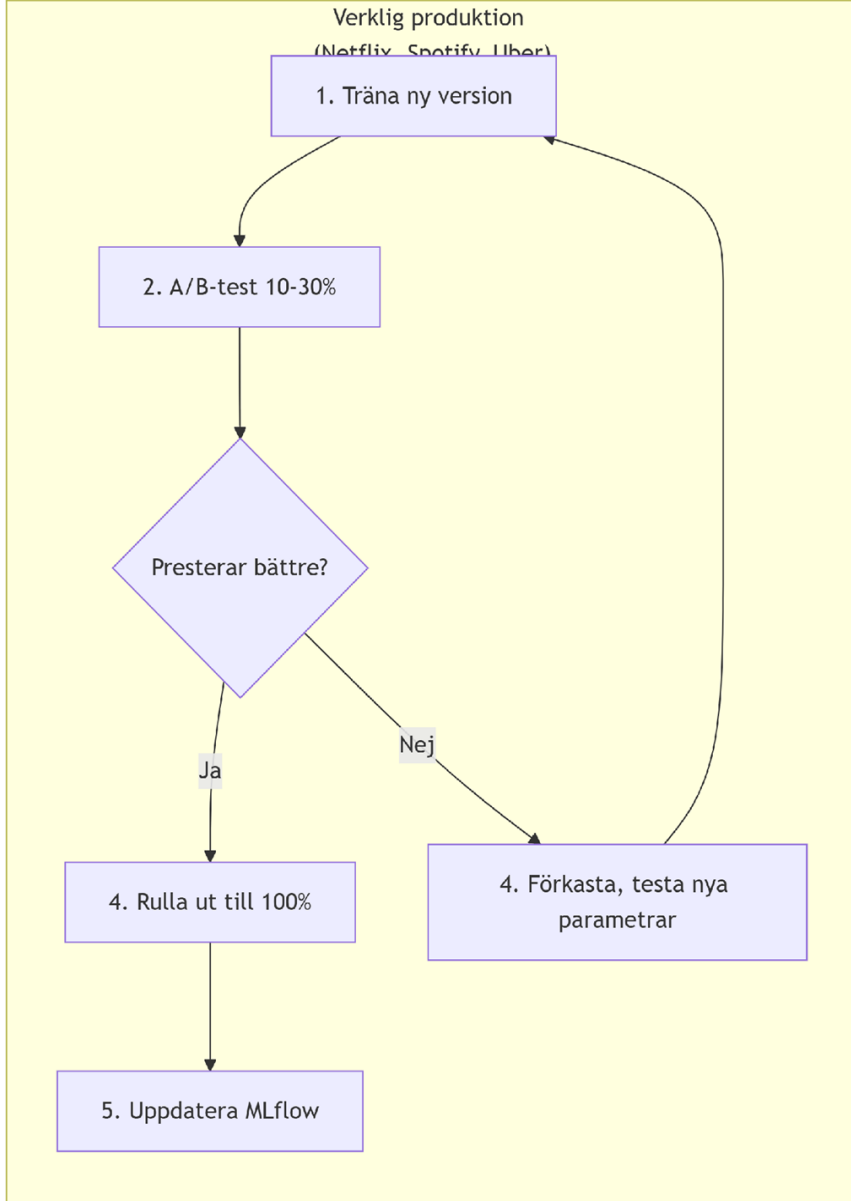
Live A/B-test (100 simuleringar, 70/30)

Mätvärde	Version A (v1.0)	Version B (v1.1)	Skillnad
Trafikandel	71%	29%	Enligt plan
Korrekthet	97.2% (69/71)	89.7% (26/29)	+7.5%
Rätt/fel	69 rätt, 2 fel	26 rätt, 3 fel	A gör färre fel

Slutsats och rekommendation

Fråga	Svar
Vilken version vann?	Version A (v1.0) – din ursprungliga modell
Varför?	Högre precision (99.1%) och bättre live-prestanda (97.2% vs 89.7%)
Vad lärde vi oss?	Fler träd (200 vs 100) och djupare träd (15 vs 10) gav ingen förbättring – tvärtom, det orsakade överanpassning
Rekommendation	Behåll v1.0 i produktion – testa andra parametrar istället

Varför A/B-testning är viktigt i produktion



Utan A/B-test

✗ Du vet inte om nya modellen är bättre

✗ Kan rulla ut en sämre modell

✗ Gissning baserad på offline-test

✗ Riskerar användarupplevelsen

Med A/B-test

✓ Du jämför direkt på riktig användardata

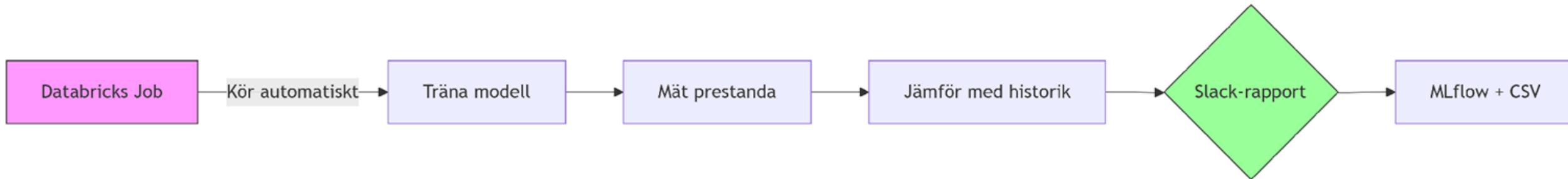
✓ Upptäcker problem innan full rollout

✓ Datadrivet beslut

✓ 30% trafik till ny modell – låg risk

Steg 12: Automatisering med Slack- rapporter & LLM- analys

- I Steg 12 bygger du en **fullständig automatiseringspipeline** som:
- **Tränar** Random Forest-modellen automatiskt
- **Mäter** prestanda (precision, recall, F1-score)
- **Jämför** med historiska modeller (via MLflow)
- **Skickar** en Slack-rapport med AI-genererad analys
- **Sparar** allt i MLflow och CSV för framtida spårning
- **Enkelt förklarat:** Du skapar ett system som självständigt tränar modellen, kontrollerar om den blir bättre eller sämre, och skickar en rapport till Slack – precis som på riktiga företag.



Verkliga exempel från arbetsmarknaden

- **Gemensamt för alla:** De kör sina modeller **automatiskt** och skickar **rapporter** till teamet – precis som du gör här!

FÖRETAG	VAD DE AUTOMATISERAR	VARFÖR
Spotify	Rekommendationsmodeller varje natt	"Vilka låtar ska vi rekommendera imorgon?"
Uber	ETA-modeller varje timme	"Hur lång tid tar resan just nu?"
Netflix	Tittarpreferensmodeller dagligen	"Vilka filmer ska vi visa på startsidan?"
Banker	Bedrägerimodeller i realtid	"Är denna transaktion misstänkt?"
Ericsson	Nätverksavvikelser varje minut	"Finns det ett pågående intrång?"

Slack vs Teams – för professionell användning

Slutsats: För skolprojekt och mindre team är **Slack mycket enklare!**

Funktion	Slack	Microsoft Teams
Webhook-inställning	✓ 2 minuter	⚠ 10+ minuter (kräver ofta admin)
Gratis	✓ Ja (gratis workspace)	✓ Ja (men ofta begränsad i skolor)
Dokumentation	✓ Utmärkt, många exempel	⚠ Krånglig, ofta föråldrad
API-anrop	✓ Enkel requests.post()	⚠ Kräver ofta Power Automate
Används i startup	✓ 90% av tech-startups	✗ Sällan
Används i storföretag	⚠ Vissa	✓ Vanligast
Push-notiser till mobil	✓ Ja	✓ Ja

Så här skapar du en Slack Webhook (steg-för-steg)

Steg	Åtgärd	Detaljer
1	Gå till api.slack.com/apps	Skapa ett Slack-konto om du inte har
2	Klicka "Create New App" → "From scratch"	Välj "From scratch" (inte från mall)
3	App Name: Insider Threat Alerts	Välj ett beskrivande namn
4	Välj workspace	Välj den workspace där din kanal finns
5	Klicka "Create App"	Appen skapas
6	I vänstermenyn, klicka "Incoming Webhooks"	Detta är vad du behöver
7	Aktivera "Activate Incoming Webhooks"	Slå på knappen
8	Klicka "Add New Webhook to Workspace"	Välj vilken kanal som ska få meddelanden
9	Välj kanal (t.ex. #all-insider-threat-detection)	Välj den kanal du skapat
10	Klicka "Allow"	Bekräfta
11	Kopiera Webhook URL:en	Den ser ut som https://hooks.slack.com/services/T.../B.../ ...

Steg 12: Komplett tabell

Steg	Åtgärd	Kod/Plats	Förklaring
12.1	Fråga efter API-nycklar	Terminalen	Slack URL, Groq API, Snowflake lösenord
12.2	Hämta data från Snowflake	<code>pd.read_sql()</code>	4 000 rader från FCT_FEATURES
12.3	Träna Random Forest	<code>RandomForestClassifier()</code>	100 träd, djup 10
12.4	Beräkna prestanda	<code>precision_score()</code> , <code>recall_score()</code>	99.1% precision, 96.7% recall
12.5	Hämta historik från MLflow	<code>mlflow.search_runs()</code>	Senaste F1-score: 0.979
12.6	Beräkna förändring	$((f1 - \text{latest_f1}) / \text{latest_f1}) * 100$	+0.00% stabil
12.7	Generera AI-analys	Groq Llama 3.3	5 meningars analys
12.8	Skicka Slack-rapport	<code>requests.post(slack_webhook_url)</code>	Meddelande med emojis och struktur
12.9	Logga till MLflow	<code>mlflow.log_metric()</code> , <code>mlflow.log_model()</code>	Sparar modell och metrics
12.10	Spara CSV-historik	<code>pd.to_csv()</code>	Alla körningar sparas

Hur detta används i verklig produktion

Företag

Spotify

Uber

Airbnb

Banker

Vad de automatiserar

Rekommendationsmodeller

ETA-prediktioner

Pris-prediktioner

Bedrägeridetektering

Vilket verktyg

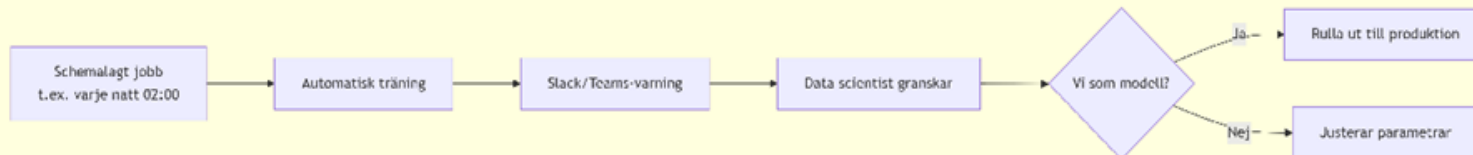
Databricks Jobs + Slack

MLflow + Slack

Databricks + Slack

MLflow + Teams

Verklig produktion (företag)



Din automation (Steg 12)

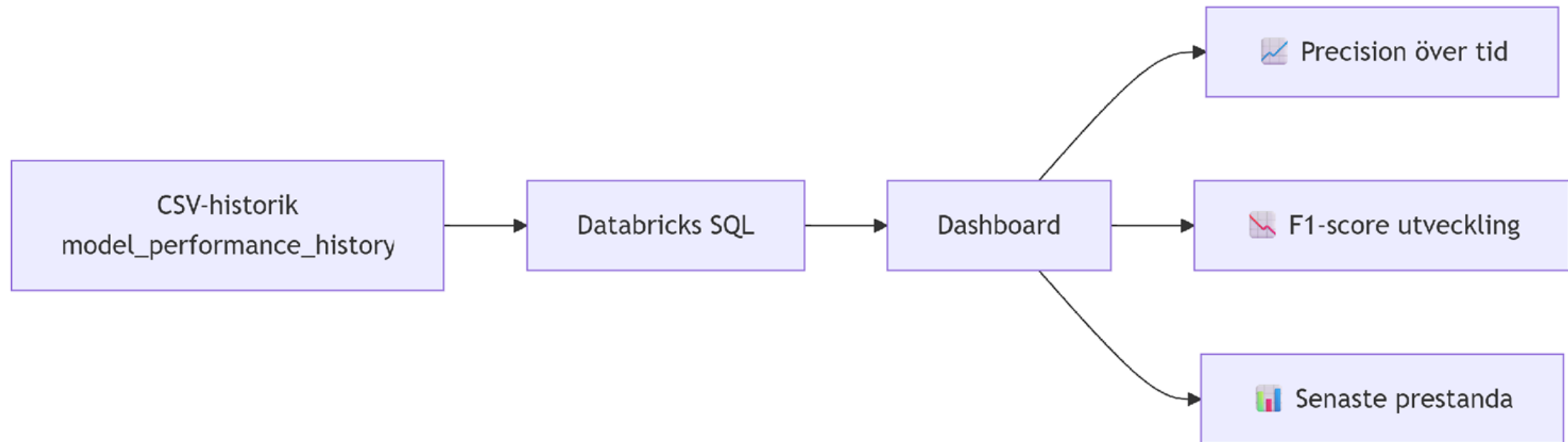


Checklista – Steg 12 är klart när:

Kontroll	☒ Klart?
Du har skapat en Slack-app och webhook URL	☐
Du har kört koden och fått ☒ Slack-rapport skickad!	☐
Du har sett meddelandet i Slack-kanalen	☐
Du har sett AI-analysen från Groq i Slack	☐
Du har sett att historiken uppdaterades (4 körningar)	☐
Du har förstått hur detta används i verkligheten	☐

Steg 13: Bygg enkel dashboard i Databricks

- I Steg 13 bygger du en **dashboard** i Databricks SQL som visualiserar din modells prestanda över tid. Detta gör att du och ditt team snabbt kan se om modellen fungerar som den ska.
- **Enkelt förklarat:** Du skapar ett "kontrollrum" där du kan se grafer över modellens precision, recall och F1-score – precis som en bilinstrumentbräda visar hastighet och bränslenivå.



Steg 13:

Komplett tabell

Steg	Åtgärd	Var	Detaljerad instruktion	Varför behövs detta?
13.1	Skapa permanent SQL-tabell	Databricks notebook	Kör Python-koden ovan	Gör historiken tillgänglig för SQL Warehouse
13.2	Skapa dataset i SQL Editor	Databricks SQL → "New Query"	Klistra in SQL-frågan	Skapa dataunderlag för dashboard
13.3	Skapa dashboard	Databricks SQL → "Dashboards"	Klicka "Create Dashboard"	Samla alla visualiseringar
13.4	Skapa linjediagram	Dashboard → "Add Visualization"	Visa prestanda över tid	Se trender och utveckling
13.5	Skapa kortvy	Dashboard → "Add Visualization"	Visa senaste värden	Snabb överblick av nyckeltal
13.6	Skapa tabell	Dashboard → "Add Visualization"	Visa alla körningar	Detaljerad historik

Resultat från din
körning.
Resultat från din
körning

MÄTVÄRDE	DITT RESULTAT	BETYDELSE
Antal körningar	4 historiska körningar	Du har spårat modellen över tid
Precision	99.1%	När modellen flaggar – har den rätt 99 gånger av 100
Recall	96.7%	Modellen hittar 97 av 100 attacker
F1-score	0.979	Utmärkt balans mellan precision och recall
Status	 STABIL	Ingen försämring sedan förra körningen

Din dashboard
kommer att
innehålla:

Visualisering	Typ	Syfte
Prestanda över tid	Linjediagram	Se trender – blir modellen bättre eller sämre?
Senaste F1-score	Kortvy (Counter)	Snabb överblick av nuvarande prestanda
Historik-tabell	Tabell	Alla körningar med detaljer

Pro tips

Tips

Använd AI-assistenten

Spara dashboarden

Dela med teamet

Uppdatera data

Förklaring

Det enklaste sättet – skriv bara vad du vill ha

Den sparas automatiskt i molnet

Klicka "Share" för att ge andra åtkomst

Kör Steg 13.1 igen efter nya körningar

Checklista – Steg 13 är klart när:



Du har kört Python-koden och skapat permanent tabell



Du har skapat dataset i SQL Editor



Du har skapat en dashboard



Dashboard har linjediagram över tid



Dashboard har kortvy med senaste F1-score

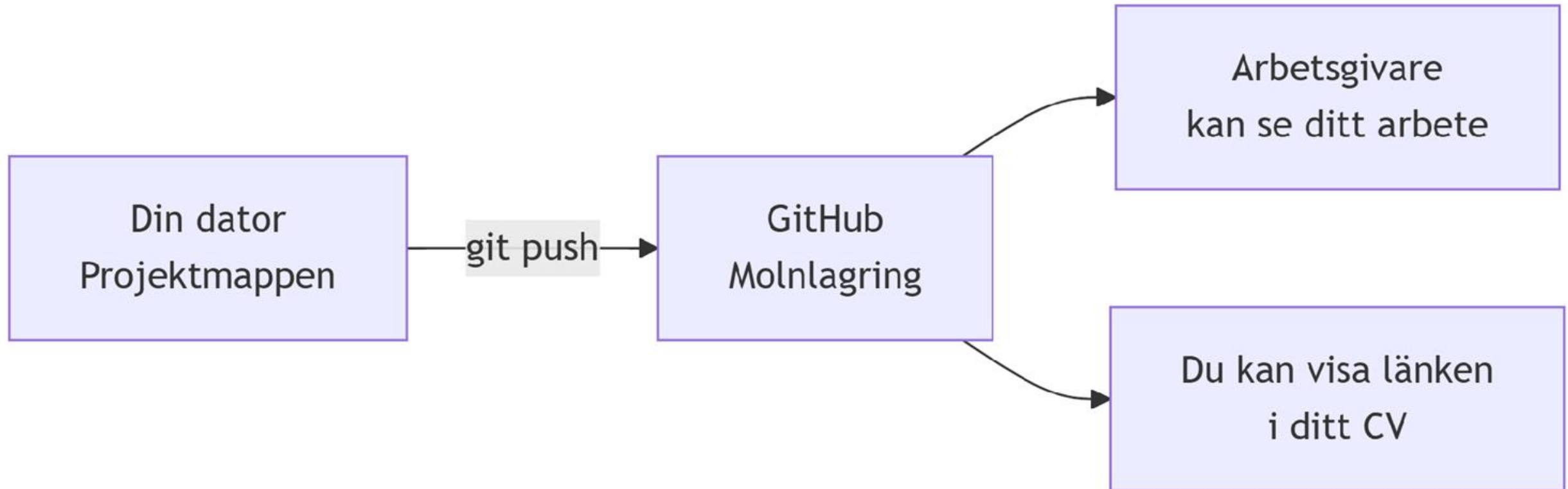


Dashboard har tabell med alla körningar



Steg 14: Ladda upp projekt till GitHub

- I Steg 14 laddar du upp **hela ditt projekt** till GitHub. Detta är viktigt för din portfolio – arbetsgivare kan se och granska ditt arbete.
- **Enkelt förklarat:** Du lägger ditt projekt i ett "digitalt skåp" (GitHub) där alla kan se det – precis som att ladda upp en



Verkliga exempel från arbetsmarknaden

Företag	Vad de laddar upp till GitHub	Varför?
Spotify	ML-modeller, rekommendationssystem	Samarbete, versionshantering
Uber	ETA-prediktioner, kartdata	Spåra ändringar, återställa vid fel
Netflix	Rekommendationsalgoritmer	Dela kunskap inom teamet
Banker	Bedrägeridetekteringsmodeller	Revisionsspår, compliance

Gemensamt för alla: De laddar **INTE** upp själva datan eller API-nycklar – bara **koden**!

Steg 14: Komplett tabell

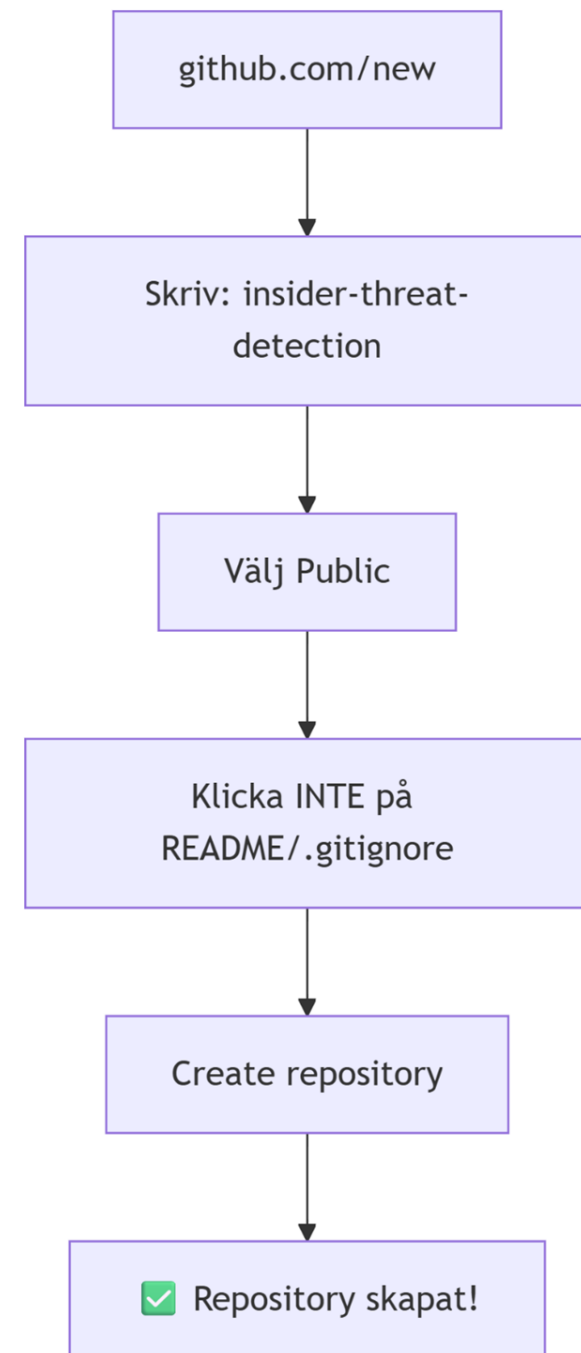
#	Steg	Åtgärd	Var	Varför?
1	Skapa GitHub-konto	Gå till github.com → "Sign up"	Webbläsare	Behövs för att lagra kod
2	Skapa nytt repository	Klicka "New repository"	GitHub webbplats	En mapp för ditt projekt
3	Namnge repository	insider-threat-detection	GitHub	Identifierar ditt projekt
4	Välj Public/Private	Välj "Public"	GitHub	Arbetsgivare kan se
5	Klicka INTE på README/.gitignore	Lämna alla rutor TOM	GitHub	Skriptet skapar dem själv
6	Klicka "Create repository"	Blå knapp	GitHub	Repositoryt skapas
7	Öppna PowerShell i projektmappen	Högerklicka i mappen → "Open in Terminal"	VS Code / PowerShell	Kör kommandon
8	Kör setup-skriptet	.\setup_github.ps1	PowerShell	Gör allt automatiskt
9	Ange GitHub-användarnamn	Skriv ditt användarnamn	Terminal	Kopplar till rätt konto
10	Välj branch-namn	master eller main	Terminal	Välj standard
11	Vänta medan skriptet körs	Automatiskt	PowerShell	Skapar .gitignore, Git LFS, commit, push
12	Kontrollera på GitHub	Gå till din repo-länk	Webbläsare	Se att alla filer kommit med

Steg 14.1: Skapa GitHub-konto (om du inte har)

Steg	Åtgärd
1	Gå till github.com
2	Klicka på "Sign up"
3	Ange e-post, lösenord och användarnamn
4	Verifiera din e-post
5	Välj "Free" plan

Steg 14.2: Skapa nytt repository på GitHub

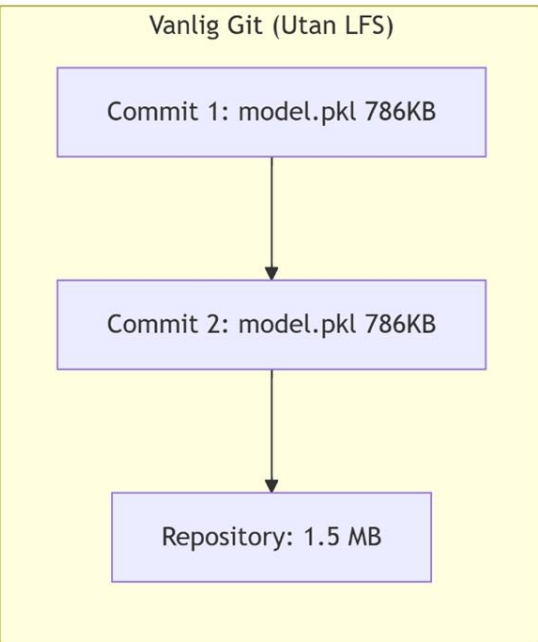
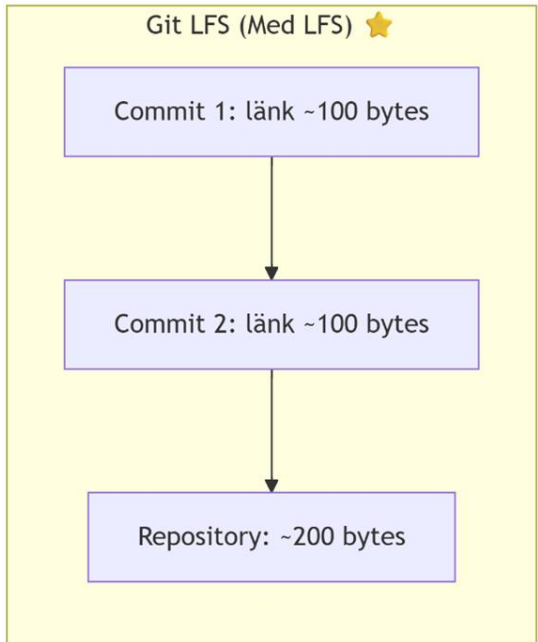
n	Åtgärd
1	Inloggad på GitHub, klicka på "+" uppe till höger
2	Välj "New repository"
3	Repository name: insider-threat-detection
4	Description: ML-projekt för insiderhot-detektering med Random Forest
5	Välj "Public" (så arbetsgivare kan se)
6	⚠ VIKTIGT: Klicka INTE på "Add a README"
7	⚠ VIKTIGT: Klicka INTE på "Add .gitignore"
8	⚠ VIKTIGT: Klicka INTE på "Choose a license"
9	Klicka på "Create repository"



Git LFS vs Vanlig Git – Varför behövs det?

Fråga	Svar
Vad är problemet med vanlig Git?	Vanlig Git sparar hela filen i historiken. Varje gång du ändrar en 1 MB fil, läggs 1 MB till i repositoryt.
Vad gör Git LFS?	Git LFS sparar bara en länk (pekare) till filen. Den stora filen lagras separat.
Varför behöver vi det?	Din .pkl-modell är 786 KB. Utan Git LFS skulle repositoryt växa snabbt. Med Git LFS förblir det litet.
Behöver arbetsgivaren Git LFS?	Ja, för att ladda ner .pkl-filerna. Men de flesta har det redan installerat.

Scenario	Utan Git LFS	Med Git LFS
Första push	786 KB	~200 bytes (länk)
10 versioner av modellen	7.8 MB	~2 KB
Att kлона repot	Laddar ner 7.8 MB	Laddar ner ~2 KB + modellen vid behov
GitHub-gräns	Max 100 MB per fil	Inga problem



Steg 14.4: Kör setup-skriptet .\setup_github.ps1. Skriptet frågar dig om:

Fråga	Vad du ska svara
Ange ditt GitHub-användarnamn	Ditt användarnamn (t.ex. Ars1222022)
Välj branch-namn	1 för main, 2 för master (välj 2 om du är osäker)
Vill du rensa och börja om?	j (ja) om du vill börja fräscht
Vill du ersätta .gitignore?	j (ja)
Har du installerat Git LFS?	j (ja) om du redan gjort det

Förklaring av setup_github.ps1 (vad koden gör)

Del i skriptet	Vad den gör	Varför?
DEL 1	Förklarar vad Git LFS är	Så att du förstår varför det behövs
DEL 2	Frågar efter GitHub-användarnamn och branch	Kopplar till rätt konto
DEL 3	Initierar Git (git init)	Skapar ett Git-repository lokalt
DEL 4	Skapar .gitignore	Ignorerar hemligheter och temporära filer
DEL 5	Installerar/konfigurerar Git LFS	Hanterar stora .pkl-filer
DEL 6	Rensar onödiga filer (manifest.mf)	Tar bort onödiga filer
DEL 7	Lägger till alla filer (git add .)	Förbereder filerna för commit
DEL 8	Skapar commit (git commit)	Sparar en version lokalt

Checklista – Steg 14 är klart när:

Kontroll	 Klart?
Du har skapat GitHub-konto	☐
Du har skapat repository insider-threat-detection	☐
Du har tagit bort API-nycklar från notebooks	☐
Du har kört .\setup_github.ps1	☐
Du ser  Push lyckades!	☐
Du kan se alla filer på GitHub-webbplatsen	☐
Länken fungerar: github.com/DITT_ANVÄNDARNAMN/insider-threat-detection	☐

Pro tips

Tips	Förklaring
Använd getpass() i framtiden	password = getpass("Ange lösenord: ") sparar INTE lösenordet i filen
Kör skriptet flera gånger	Det skapar inga dubbletter – bara lägger till ändringar
Kontrollera GitHub efter push	Gå till din repo-länk och se att alla filer finns
Dela länken	github.com/DITT_ANVÄNDARNAMN/insider-threat-detection – lägg i ditt CV!

Steg 14 är klart – projektet är KLART!

